

Comparing Verbal and Non-Verbal Communication Styles in an LLM-Powered Accountability Robot

Omar Ankit

Department of Electrical and Computer Engineering
University of Waterloo
Waterloo, Ontario, Canada
oankit@uwaterloo.ca

Abstract—This paper investigates how different communication styles affect user perception of an LLM-powered accountability coaching robot. We implemented three communication styles (sarcastic, motivational, and neutral) on the Reachy Mini robot using ElevenLabs Conversational AI agents. A key technical contribution is a weighted arbitration system for non-verbal behavior selection, inspired by Rosenblatt and Payton’s DAMN architecture (1989), where personality-specific weight multipliers boost or suppress detected emotion-keyword behaviors across independent channels (emotion, dance, head movement). A within-subjects video-based study (N=39) revealed that the motivational style scored highest on likability (M=3.46), appropriateness ($p<0.01$), and perceived warmth ($p<0.001$), while the sarcastic style scored highest on entertainment (M=3.69, $p=0.020$). Despite lowest likability, 33% of participants preferred the sarcastic robot as their daily coach, suggesting that entertainment value may drive preference more than traditional likability metrics. Demographic analysis revealed that younger participants found sarcasm significantly more entertaining ($p=0.026$), women found it significantly more uncomfortable ($p=0.023$), and infrequent AI users preferred it overall. Non-verbal expressiveness had a moderate effect on perception (M=3.08/5 importance), with the motivational condition rated highest on body-movement congruence.

Index Terms—Human-Robot Interaction, Social Robotics, Communication Styles, LLM, Non-Verbal Behavior, Accountability Coaching, Weighted Arbitration

I. INTRODUCTION

Large Language Models (LLMs) have enabled robots to hold natural, open-ended conversations with users. However, for accountability coaching, a context requiring both positive reinforcement and constructive criticism, the manner in which feedback is delivered may matter as much as its content. Communication style encompasses both the verbal strategies (word choice, tone, humor) and non-verbal behaviors (gestures, head movements, body language) that shape how a message is perceived.

Prior work in Human-Robot Interaction (HRI) has established that multimodal expression outperforms verbal-only communication [1], that expressive robot tutors improve learning outcomes [3], and that LLM-driven personality traits affect user emotional states [5]. Recent work has also shown that even on non-anthropomorphic robots, expression-driven movements significantly enhance user engagement, particularly in social tasks [7]. However, few studies compare multiple dis-

tinct LLM-driven communication styles with coupled non-verbal behavior on a physical robot.

This project addresses that gap by implementing three communication styles on the Reachy Mini desktop robot: sarcastic (a sharp-witted character using dry humor and back-handed compliments), motivational (a relentlessly positive hype coach), and neutral (a flat, informational assistant akin to Siri or Alexa). Each style is realized through a unique ElevenLabs Conversational AI agent with a keyword-aware system prompt that controls both the LLM’s word choice and, through a novel weighted arbitration mechanism, the robot’s non-verbal body language.

We conducted a within-subjects video-based study (N=39) to evaluate how these styles affect perceived likability, entertainment value, motivation, warmth, and body-movement congruence. Our results reveal a tension between likability and preference: the motivational robot scored highest on traditional metrics, but 33% of participants preferred the sarcastic robot as their daily coach, driven by its entertainment value. Demographic analysis further shows that age, gender, and AI familiarity significantly moderate these preferences.

II. RELATED WORK

A. Multimodal Expression in HRI

Aly and Tapus [1] demonstrated that robots expressing personality through both verbal and non-verbal channels are perceived more positively than those using verbal strategies alone. Salem et al. [2] showed that co-verbal gestures increase perceived anthropomorphism and likability. Saerbeck et al. [3] found that an expressive robot tutor improved learning outcomes compared to a neutral one, establishing that expressiveness matters in educational and coaching contexts.

Lee and Hahn [4] investigated head movements on robots and found that pre-utterance movements increase perceived likability and naturalness, suggesting that the timing of non-verbal behavior relative to speech affects perception.

B. LLM-Driven Robot Communication

Recent work has explored using LLMs to generate and adapt robot communication styles. Zhang et al. [5] implemented LLM personality traits on the Furhat robot and found they affect user emotional states. Mishra et al. [6] used LLMs to generate tagged non-verbal behaviors for the QTrobot,

demonstrating that LLM output can drive robot body language through behavior tags embedded in the generated text.

C. Expressive Movement Design

Hu et al. [7] proposed the ELEGNT framework for designing expressive movements on non-anthropomorphic robots. Their user study (N=21) comparing expression-driven versus function-driven movements on a lamp-like robot showed that expressive movements significantly enhanced engagement, perceived character, and sense of connection, especially in social-oriented tasks. This finding supports our hypothesis that coupling communication style with expressive body language should enhance the coaching interaction.

D. Behavior Arbitration Architectures

Multiple architectures exist for coordinating competing robot behaviors. Brooks’ subsumption architecture [9] uses fixed-priority layers, while Maes’ activation networks [10] use spreading activation. Rosenblatt and Payton’s Distributed Architecture for Mobile Navigation (DAMN) [8] introduced weighted voting where multiple behaviors propose actions and context-dependent weights determine the winner. We adopt a DAMN-inspired approach because the weights directly encode personality: the sarcastic personality boosts certain behaviors while suppressing others, making the arbitration mechanism itself an expression of communication style.

III. RESEARCH QUESTIONS AND HYPOTHESES

Our study is guided by two research questions:

RQ1: How does communication style affect perceived likability, appropriateness, and entertainment value of an accountability coaching robot?

RQ2: Does non-verbal expressiveness contribute to how participants perceive each communication style?

Based on prior findings that positive feedback is accepted more readily [11] and that humor increases entertainment but may reduce perceived warmth [12], we formulate three hypotheses:

H1: The motivational style will score highest on likability and perceived warmth.

H2: The sarcastic style will score highest on entertainment value.

H3: The motivational style will receive the highest body-movement congruence ratings due to having the most physical movement.

IV. ROBOTIC IMPLEMENTATION

A. Platform: Reachy Mini

The Reachy Mini is a 9-degree-of-freedom desktop robot manufactured by Pollen Robotics. It features a 6-DOF Stewart platform head (enabling expressive tilts, nods, and shakes), two independently controlled antennas, and a 360° rotating body. The robot includes a built-in microphone and speaker for voice interaction. Its compact form factor and expressive head make it suitable for social interaction research.



Fig. 1: The Reachy Mini desktop robot used in this study, shown in a typical desk-based interaction setting.

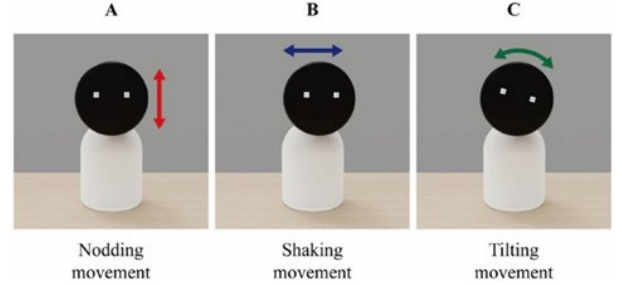


Fig. 2: The three primary head movement primitives of the Reachy Mini: (A) nodding (vertical), (B) shaking (horizontal), and (C) tilting (rotational). These movements form the basis of the robot’s non-verbal communication repertoire.

B. Voice Pipeline: ElevenLabs Conversational AI

Each communication style is implemented as a separate ElevenLabs Conversational AI agent, selected via an environment variable (ELEVENLABS_AGENT_ID). The ElevenLabs platform provides an end-to-end voice pipeline: Voice Activity Detection (VAD) detects when the user is speaking, automatic speech recognition transcribes the speech, the LLM generates a response, and text-to-speech synthesizes the output audio. This pipeline runs remotely, with audio streamed to and from the robot in real time.

C. Keyword-Aware System Prompts

The key design insight is that we control the robot’s body language indirectly by controlling the LLM’s word choice. Each agent has a system prompt with three sections:

Personality: Defines the character. The sarcastic agent is “Megatron,” a sharp-witted character trapped in a small robot body. The motivational agent is a relentlessly positive hype coach. The neutral agent is a calm, professional assistant with no personality.

Environment: A hardcoded JSON schedule and user profile, identical across all three agents, ensuring the coaching scenario is controlled.

Expressive Language Rules: USE/AVOID keyword rules that determine which words the LLM incorporates. The sarcastic prompt instructs: USE words like “ridiculous,” “nonsense,” “shocking” (which trigger reprimand, irritated, and surprised

TABLE I: Personality Weight Multipliers (Selected Behaviors)

Behavior	Sarcastic	Motivational	Neutral
reprimand	$\times 1.5$	$\times 0.3$	$\times 0.1$
enthusiastic	$\times 0.4$	$\times 1.5$	$\times 0.1$
dance	$\times 0.3$	$\times 1.5$	$\times 0.0$
happy	$\times 0.5$	$\times 1.3$	$\times 0.1$
irritated	$\times 1.3$	$\times 0.4$	$\times 0.1$
proud	$\times 0.5$	$\times 1.4$	$\times 0.1$
greeting	—	—	$\times 0.8$

body language) and AVOID words like “amazing,” “incredible” (which would trigger celebratory body language inconsistent with the sarcastic persona). The motivational prompt does the inverse. The neutral prompt avoids all emotional trigger words, keeping the robot mostly still.

This approach is inspired by Mishra et al.’s [6] technique of embedding behavior tags in LLM output, but rather than using explicit tags, we leverage natural language keywords that the LLM incorporates organically into its responses.

D. System Architecture Overview

Fig. 3 illustrates the end-to-end system architecture. User speech is captured by the Reachy Mini’s microphone and streamed to the ElevenLabs Conversational AI platform, which handles voice activity detection, speech recognition, LLM generation, and text-to-speech synthesis. The generated text is simultaneously forwarded to our on-device Emotion Detector, which scans for keyword patterns and computes per-category confidence scores. These scores are then passed through the personality-weighted arbitration layer, where DAMN-inspired weight profiles amplify or suppress behaviors according to the active communication style. The resulting behavior commands are fed to the Movement Manager, which composes emotion-driven poses with audio-reactive head wobble to produce the final motor commands.

E. Emotion Detection with Weighted Arbitration

The EmotionDetector module scans the LLM’s text output for approximately 50 keyword patterns across categories including happy, enthusiastic, reprimand, irritated, surprised, confused, greeting, and dance triggers. Each pattern has associated keywords (e.g., “ridiculous,” “nonsense,” “foolish” for reprimand) and regex patterns (e.g., emoji detection).

1) *Scoring*: For each keyword match, a confidence score is computed based on keyword length (longer keywords are more specific) and pattern matches. Scores are accumulated per emotion category and capped at 1.0.

2) *Personality-Weighted Arbitration*: Inspired by Rosenblatt and Payton’s DAMN architecture [8], raw confidence scores are multiplied by personality-specific weight profiles. Table I shows representative weights. This design means the weights are the personality. The sarcastic profile boosts negative/mocking behaviors and suppresses positive ones; the motivational profile does the opposite; the neutral profile suppresses almost everything, ensuring minimal movement.

3) *Per-Category Selection and Execution*: After weighting, candidates are grouped by action category: `play_emotion`, `dance`, and `move_head`. The highest-scoring candidate in each category is selected independently. This allows complementary behaviors to fire simultaneously from a single response, for example, a reprimand emotion paired with a head shake, or an enthusiastic emotion paired with a dance. Each category has an independent cooldown timer to prevent repetitive actions.

F. Movement Manager

The MovementManager runs a 100Hz control loop that composes multiple motion sources into a single `set_target()` call per tick:

Primary poses from the emotion/dance queue (sequential, mutually exclusive).

Secondary offsets from the HeadWobbler (audio-reactive lip-sync sway computed from TTS audio deltas).

Primary and secondary poses are fused using `compose_world_offset()`, which applies the secondary offset in world-frame coordinates to the primary pose. Antenna positions are frozen during listening mode and blended back smoothly when the robot responds.

G. Idle Emotion System

An IdleEmotionManager runs a background thread that queues subtle emotions (e.g., attentive, thoughtful, curious, proud) between agent responses. It only fires when no move is currently playing and no moves are queued, preventing interference with conversation-driven emotions.

H. End-to-End Pipeline Walkthrough

To illustrate how the system components interact, we trace a single exchange from the sarcastic condition through the full pipeline.

Step 1: User input. The user says: “I skipped my LeetCode practice today.” The Reachy Mini’s microphone captures the audio and streams it to the ElevenLabs agent.

Step 2: LLM generation. The ElevenLabs pipeline transcribes the speech, and the sarcastic agent’s system prompt (“Megatron”) processes it. The prompt’s USE rules instruct the LLM to incorporate words like “ridiculous” and “nonsense” when the user fails. The agent generates: “You skipped LeetCode and still called it a win? Ridiculous.”

Step 3: Parallel output. The generated text is sent simultaneously to (a) the TTS module, which synthesizes audio streamed back to the robot’s speaker, and (b) the on-device Emotion Detector.

Step 4: Keyword scoring. The Emotion Detector scans the text against ~ 50 keyword patterns. “Ridiculous” matches the reprimand category (keywords: “ridiculous,” “nonsense,” “foolish”), yielding a raw confidence of $\frac{10}{20} = 0.50$ (based on keyword length). No other strong matches are found.

Step 5: Personality-weighted arbitration. The sarcastic weight profile multiplies the raw score: reprimand $0.50 \times 1.5 = 0.75$. Had the agent instead used a celebratory word like “incredible” (which the AVOID rules suppress), the enthusiastic

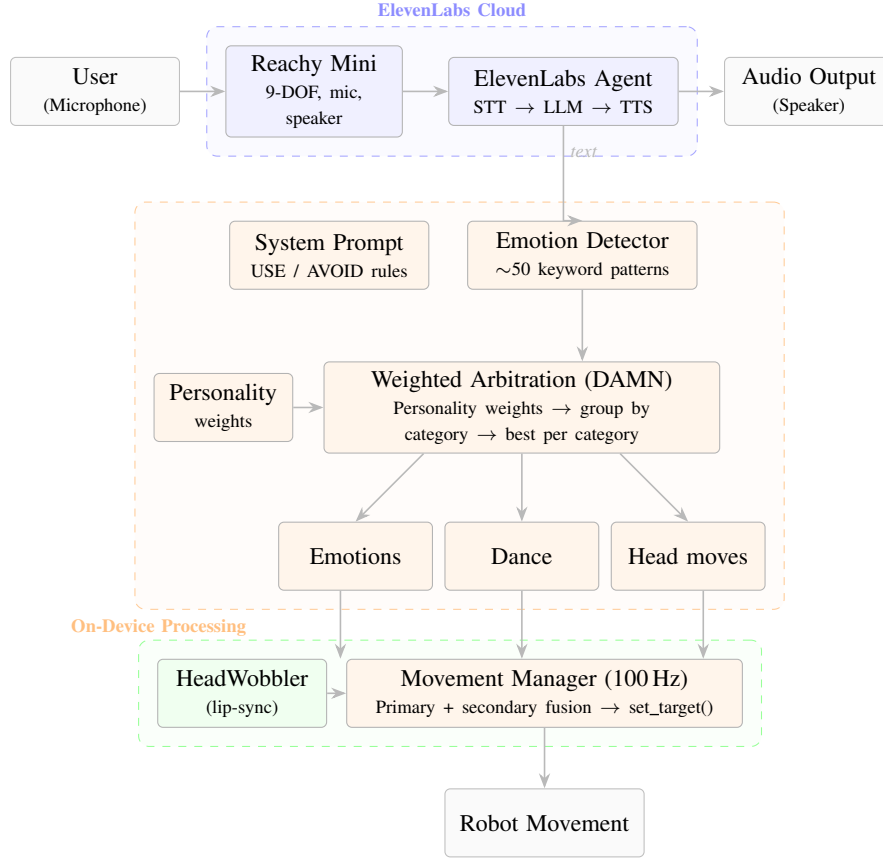


Fig. 3: End-to-end system architecture. User voice input is captured by the Reachy Mini and streamed to an ElevenLabs Conversational AI agent (STT → LLM → TTS). The style-specific system prompt controls keyword usage via USE/AVOID rules. The Emotion Detector scans the text response for ~50 keyword patterns, and the DAMN-inspired Weighted Arbitration layer applies personality weights to select the best behavior per category (emotions, dance, head moves). The Movement Manager fuses these primary behaviors with secondary lip-sync offsets from the HeadWobbler at 100 Hz to produce the final robot movement.

category would score $0.50 \times 0.4 = 0.20$, falling below the 0.3 confidence threshold. This is how the weights enforce personality-congruent behavior.

Step 6: Per-category selection. Candidates are grouped by action category. In this case, only `play_emotion` has a candidate above threshold (reprimand at 0.75). No dance or `move_head` candidates survive weighting.

Step 7: Execution. The reprimand emotion is queued in the Movement Manager. Simultaneously, the HeadWobbler receives the TTS audio stream and computes audio-reactive lip-sync sway offsets.

Step 8: Pose fusion. The Movement Manager’s 100 Hz loop samples the reprimand emotion (primary pose: a sharp forward lean with antenna tilt) and composes it with the HeadWobbler’s sway offsets (secondary pose) via `compose_world_offset()`, producing a single `set_target()` call per tick.

The result: the robot delivers the sarcastic line through its speaker while simultaneously performing a reprimand gesture with subtle audio-reactive head movement, creating a coupled

verbal and non-verbal response that is congruent with the sarcastic personality.

I. System Validation

Interaction logs from each condition confirm the pipeline operates as designed across all three personalities:

Motivational: “Incredible! You nailed it! Let’s celebrate!” → enthusiastic ($0.50 \times 1.5 = 0.75$) + dance celebrate ($0.45 \times 1.5 = 0.68$) fire simultaneously from separate categories.

Sarcastic: “You skipped LeetCode and still called it a win? Ridiculous.” → reprimand ($0.50 \times 1.5 = 0.75$) fires; enthusiasm is suppressed by the $\times 0.4$ weight.

Neutral: “Task marked as complete. Next item: Leetcode.” → success ($0.40 \times 0.1 = 0.04$) falls below threshold; robot remains mostly still, consistent with the neutral profile’s near-zero weights.

V. METHOD

A. Overall Design

We conducted a within-subjects video-based study with three conditions (sarcastic, motivational, neutral). The order of



Fig. 4: Examples of expressive poses on the Reacy Mini. Left: “Curious” expression with a tilted head. Right: “Dance” expression with rotated body orientation. These poses are triggered by the weighted arbitration system based on detected keywords.

conditions was counterbalanced using a Latin Square design with three form versions distributed via Google Forms.

B. Participants

Thirty-nine participants (24 male, 15 female) were recruited from the ECE 787 course cohort at the University of Waterloo. Ages ranged from 21 to 60+, with the majority (27/39) aged 30 or older. Most participants (32/39) reported using AI assistants daily or weekly.

C. Scenario

Each condition used the same three-beat scenario to elicit success feedback, failure feedback, and advice:

- 1) **Success:** “I just finished my ECE 787 midterm.”
- 2) **Failure:** “I skipped my LeetCode practice today.”
- 3) **Advice:** “What should I do for the rest of the day?”

This design ensures each communication style demonstrates how it handles praise, criticism, and guidance within the same conversational context.

D. Procedure

Participants first completed a pre-study questionnaire with demographic items and three NARS-inspired items [13] measuring comfort with robot feedback, unease about robot emotions, and concern about robot dependence. They then watched three video recordings of the robot interacting in the coaching scenario (one per condition), rating each on a per-condition questionnaire before proceeding to the next video.

After all three conditions, participants completed a comparison questionnaire with forced-choice and open-ended items.

E. Measures

Godspeed Likability [11]: Five semantic differential items (Dislike–Like, Unfriendly–Friendly, Unkind–Kind, Unpleasant–Pleasant, Awful–Nice) on a 1–5 scale.

Custom Likert items (5-point scale): Appropriateness for coaching, entertainment value, motivation to complete tasks, body-movement match, and willingness to use daily.

RoSAS [12] (trimmed to 7 items): Warmth subscale (Happy, Compassionate, Emotional), Competence subscale (Capable, Responsive), Discomfort subscale (Awkward, Aggressive).

Post-comparison: Preferred daily coach (forced choice), most expressive body language (forced choice), most useful feedback (forced choice), body language importance (1–5), and three open-ended questions.

F. Statistical Approach

Paired-samples *t*-tests were used for within-subjects comparisons across the three conditions, with Cohen’s *d* computed for effect sizes. Independent-samples *t*-tests were used for demographic subgroup comparisons (gender, age group, AI usage frequency). Pearson correlations were computed between pre-study attitudes and key dependent variables. Significance was evaluated at $\alpha = 0.05$.

VI. RESULTS

A. Demographics

The sample comprised 24 males and 15 females, predominantly aged 30+ (27/39). AI assistant usage was high: 21 daily, 11 weekly, 6 rarely, 1 never. Pre-study attitudes showed moderate comfort with robot feedback ($M=3.56$, $SD=1.02$), moderate unease about robot emotions ($M=3.79$, $SD=1.15$), and moderate concern about dependence ($M=3.64$, $SD=1.33$).

B. RQ1: Communication Style Effects

Table II presents mean scores across all conditions. Table III presents significant paired comparisons. Figs. 5 and 6 visualize the key findings.

H1 is supported: The motivational style scored significantly higher than sarcastic on likability ($p = .017$, $d = 0.40$) and significantly higher than both other styles on appropriateness, motivation, and warmth (all $p < .01$, $d > 0.45$).

H2 is supported: The sarcastic style scored significantly higher than neutral on entertainment ($p = .020$, $d = 0.39$).

H3 is supported: The motivational style scored significantly higher than both other styles on body-movement match ($p < .01$, $d > 0.50$).

C. Forced Choice and Body Language

Motivational was preferred as daily coach by 41% (16/39), sarcastic by 33% (13/39), and neutral by 26% (10/39), as shown in Fig. 7. Motivational was rated most expressive (44%), followed by sarcastic (38%) and neutral (18%). Motivational was rated most useful (56%), followed by neutral (26%) and sarcastic (18%).

TABLE II: Summary of Mean Scores by Condition (1–5 Scale)

Measure	Neutral	Sarcastic	Motivational
Godspeed Likability	3.17	2.98	3.46
Appropriate	2.85	3.00	3.72
Entertaining	2.97	3.69	3.54
Motivating	2.72	3.08	3.72
Body-movement match	2.62	2.90	3.38
Daily use	2.62	2.97	3.18
RoSAS Warmth	2.45	2.77	3.55
RoSAS Competence	3.31	3.50	3.68
RoSAS Discomfort	2.51	2.88	2.29

TABLE III: Significant Paired t -Tests ($\alpha = 0.05$)

Comparison	Measure	p	d	H
Motiv > Sarc	Likability	.017	0.40	H1
Sarc > Neut	Entertainment	.020	0.39	H2
Motiv > Neut	Entertainment	.038	0.35	–
Motiv > Neut	Appropriate	.007	0.46	H1
Motiv > Sarc	Appropriate	.008	0.45	H1
Motiv > Neut	Motivation	<.001	0.62	H1
Motiv > Sarc	Motivation	.008	0.45	H1
Motiv > Neut	Body-move.	.001	0.57	H3
Motiv > Sarc	Body-move.	.003	0.50	H3
Motiv > Neut	Warmth	<.001	0.61	H1
Motiv > Sarc	Warmth	<.001	0.63	H1
Sarc > Motiv	Discomfort	.010	0.43	–
Sarc > Neut	Discomfort	.052	0.32	–

Body language importance was moderate ($M=3.08$, $SD=1.35$). Several participants reported ignoring body movements: “I personally felt that body movements were random” (P11); “The language used affected me more” (P37); “Body language for me is not important with a robot” (P2).

D. Demographic Discrepancies

Demographic subgroup analyses, summarized in Fig. 8, revealed significant moderating effects of gender, age, and AI usage frequency on sarcastic style perception.

Gender: Women found the sarcastic robot significantly more uncomfortable than men (F : $M=3.37$, M : $M=2.58$, $p = .023$). Women were equally split between sarcastic and motivational as preferred coach (6/6), while men favored motivational (10 vs. 7).

Age: Younger participants (<30 , $N=12$) found sarcastic significantly more entertaining than older participants (30+, $N=27$): $M=4.33$ vs. $M=3.41$, $p = .026$. Younger participants split evenly between sarcastic and motivational (5/5), while older participants favored motivational (11 vs. 8).

AI usage: Infrequent users ($N=7$) preferred sarcastic (4/7), rated it most likable ($M=3.46$) and most appropriate ($M=3.43$), completely inverting the overall pattern. Frequent users ($N=32$) preferred motivational (15/32) and were significantly more comfortable with robot feedback ($p = .041$).

E. Qualitative Findings

Open-ended responses revealed key themes:

Sarcastic felt human: “B felt more human” (P1); “Clean roasts that made it feel more human” (P17); “Aggressiveness is required for a coaching robot” (P13).

Godspeed Likability

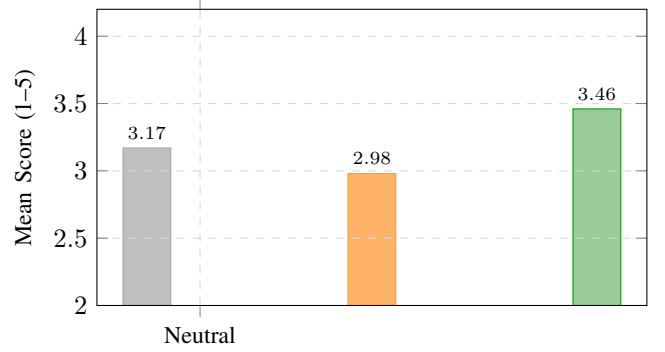


Fig. 5: Godspeed Likability scores across conditions. Motivational scored highest ($M=3.46$), followed by neutral ($M=3.17$) and sarcastic ($M=2.98$). The difference between motivational and sarcastic was significant ($p = .017$, $d = 0.40$).

Entertainment vs. Motivation

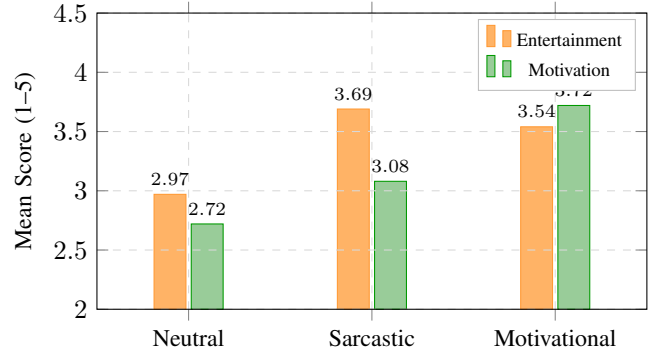


Fig. 6: Entertainment and Motivation scores by condition. Sarcastic led on entertainment ($M=3.69$, $p = .020$ vs. neutral) while motivational led on motivation ($M=3.72$, $p < .001$ vs. neutral). This illustrates the central tension: sarcastic engages through entertainment, motivational through perceived utility.

Motivational perceived as excessive: “C was somehow faking it” (P16); “Most motivational but personally feels like too much” (P17); “C too kind and not effective” (P3).

Desire for hybrid: “Liked B’s funny comments. But if it was less and combined a bit with model C’s compassion” (P4).

VII. DISCUSSION

A. RQ1: Style Affects Perception

Communication style significantly differentiates how participants perceive an accountability coaching robot. The motivational style wins on traditional HRI metrics: likability, warmth, appropriateness, and perceived motivation. This is consistent with prior findings that positive feedback is accepted more readily [11] and that expressive robots are perceived more favorably in educational contexts [3].

However, the sarcastic style reveals a tension between likability and preference that is not captured by standard instruments. Despite scoring lowest on likability (2.98) and

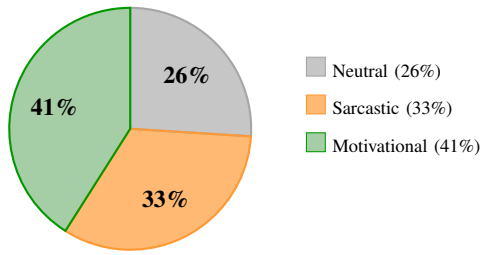


Fig. 7: Preferred daily coach distribution. Despite scoring lowest on likability, the sarcastic robot was preferred by one-third of participants, highlighting the gap between traditional acceptance metrics and user preference.

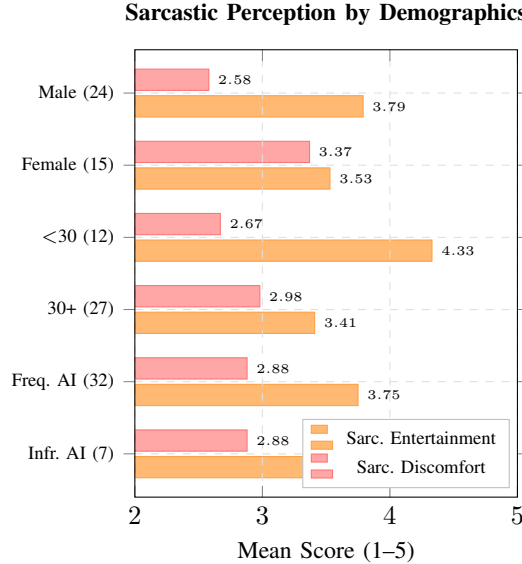


Fig. 8: Sarcastic style perception across demographic subgroups. Younger participants (<30) rated sarcasm significantly more entertaining ($p = .026$). Women reported significantly higher discomfort ($p = .023$). Infrequent AI users showed elevated entertainment comparable to frequent users.

highest on discomfort (2.88), sarcastic was rated highest on entertainment (3.69) and was chosen as the preferred daily coach by 33% of participants. This suggests that entertainment value may be a stronger driver of sustained engagement than likability, a finding with implications for coaching robot design, where long-term adherence matters more than first-impression ratings.

The qualitative data supports this interpretation: participants described the sarcastic robot as “more human” and noted that the motivational robot felt “fake” or “too much.” This echoes findings from Hu et al. [7] that excessive expressiveness without clear purpose can reduce perceived authenticity.

B. RQ2: Non-Verbal Contribution

Non-verbal expressiveness had a moderate effect on perception (body language importance $M=3.08/5$). The motivational condition received the highest body-movement congruence rat-

ings, supporting H3 and suggesting that the weighted arbitration system successfully differentiated non-verbal expression across styles. However, several participants reported ignoring body movements entirely.

This moderate result may be partially attributable to the simulation being somewhat buggy, which did not fully showcase the range of body movements alongside the emotions. Additionally, the keyword-based detection system has inherent limitations: cooldown timers prevent rapid successive emotions, and the timing of emotion triggers relative to speech may not always align naturally. Lee and Hahn’s [4] finding that pre-utterance timing increases naturalness suggests that improving the temporal alignment of non-verbal behaviors with speech could strengthen the non-verbal contribution.

Our study evaluates communication styles as coupled verbal and non-verbal packages, which mirrors how humans naturally communicate. A future study could isolate the non-verbal channel by testing the same verbal output with and without body movements, providing a cleaner test of non-verbal contribution.

C. Demographic Moderation

The demographic findings suggest that communication style preference for coaching robots should be personalized. Younger users (<30) found sarcasm significantly more entertaining and were equally likely to choose it as their daily coach. Women found sarcasm more uncomfortable but were still equally split in preference. Infrequent AI users actually preferred the sarcastic robot overall, perhaps because its distinctiveness was more novel compared to the Siri/Alexa-like neutral style they rarely encounter.

These findings align with Hu et al.’s [7] observation that age significantly influenced perceptions of expressive robots, with older participants showing less preference for expressive behaviors. Combined, these results motivate future work on adaptive style selection: choosing communication style based on user demographics and preferences, or blending styles contextually (e.g., motivational for praise, sarcastic for criticism), as suggested by participant P4.

VIII. LIMITATIONS AND FUTURE WORK

Sample and method: The video-based evaluation does not capture the dynamics of live interaction, where turn-taking, interruption handling, and real-time adaptation would affect perception differently.

No isolated non-verbal condition: Our study compares coupled verbal + non-verbal packages. Without a verbal-only control, we cannot definitively attribute perception differences to the non-verbal component alone.

Simulation fidelity: The range of body movements that could be expressed alongside the emotions was not thoroughly tested. The simulation was somewhat buggy and did not do the body movements full justice, which may explain why some participants perceived them as random.

Detection limitations: The keyword-based emotion detection has cooldown limits (one action per category per response)

and does not account for context, negation, or sarcasm in the text itself. A semantic understanding approach could improve detection accuracy.

No longitudinal data: First-impression effects may differ from sustained-use effects. The novelty of the sarcastic robot may wear off, or the motivational robot’s perceived fakeness may compound over time.

Future work: Key directions include (1) live interaction studies with real-time conversation, (2) a 3×2 design isolating verbal from non-verbal, (3) adaptive style selection based on user demographics and in-session sentiment, (4) longitudinal studies measuring task completion rates over weeks, and (5) hybrid styles that blend motivational warmth with sarcastic engagement contextually.

IX. CONCLUSION

This study demonstrates that communication style significantly affects how users perceive an LLM-powered accountability coaching robot, and that the relationship between perception and preference is more nuanced than traditional likability metrics suggest. All three hypotheses were supported: motivational wins on likability, warmth, and appropriateness; sarcastic wins on entertainment; and motivational achieves the best body-movement congruence.

The key contribution is the finding that likability does not predict preference: 33% of participants chose the least-likable robot as their preferred daily coach, driven by its entertainment value and perceived human-like quality. This tension suggests that coaching robot designers should consider engagement and entertainment alongside traditional acceptance metrics.

The technical contribution, a DAMN-inspired weighted arbitration system that couples LLM word choice to personality-congruent non-verbal behavior, provides a reusable architecture for implementing distinct communication styles on expressive robots. By controlling the LLM’s system prompt, researchers can create differentiated robot characters without modifying the underlying behavior detection or movement systems.

ACKNOWLEDGEMENTS

The author extends sincere gratitude to Professor Kerstin Dautenhahn for her expertise in social robotics and guidance throughout this project, and to all 39 participants who contributed their time and feedback to this study.

REFERENCES

- [1] A. Aly and A. Tapus, “A model for synthesizing a combined verbal and nonverbal behavior based on personality traits in human-robot interaction,” *Autonomous Robots*, vol. 41, no. 1, pp. 235–257, 2016.
- [2] M. Salem, F. Eyssel, K. Rohlfing, S. Kopp, and F. Joubin, “To err is human(-like): Effects of robot gesture on perceived anthropomorphism and likability,” *Int. J. Social Robotics*, vol. 5, no. 3, pp. 313–323, 2013.
- [3] M. Saerbeck, T. Schut, C. Bartneck, and M. D. Janse, “Expressive robots in education: Varying the degree of social supportive behavior of a robotic tutor,” in *Proc. SIGCHI Conf. Human Factors in Computing Systems*, 2010, pp. 1613–1622.
- [4] J. Lee and S. Hahn, “Effects of robot head movements on human perception: Pre-utterance movements increase likability and naturalness,” in *Proc. ACM/IEEE Int. Conf. Human-Robot Interaction (HRI)*, 2025.
- [5] Y. Zhang, M. Axelsson, and G. Hoffman, “LLM personality traits on Furhat: Exploring the effects on user emotional states,” in *Proc. ACM/IEEE Int. Conf. Human-Robot Interaction (HRI)*, 2025.
- [6] C. Mishra, P. Verma, and K. Dautenhahn, “Real-time emotion generation in human-robot dialogue using large language models,” in *Proc. IEEE Int. Conf. Robot and Human Interactive Communication (RO-MAN)*, 2023.
- [7] Y. Hu, P. Huang, M. Sivapurapu, and J. Zhang, “ELEGNT: Expressive and functional movement design for non-anthropomorphic robot,” in *Proc. ACM Designing Interactive Systems (DIS)*, 2025.
- [8] J. K. Rosenblatt and D. W. Payton, “A fine-grained alternative to the subsumption architecture for mobile robot control,” in *Proc. IEEE/INNS Int. Joint Conf. Neural Networks*, 1989, vol. 2, pp. 317–324.
- [9] R. A. Brooks, “A robust layered control system for a mobile robot,” *IEEE J. Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.
- [10] P. Maes, “How to do the right thing,” *Connection Science*, vol. 1, no. 3, pp. 291–323, 1989.
- [11] C. Bartneck, D. Kulić, E. Croft, and S. Zoghbi, “Measurement instruments for the anthropomorphism, animacy, likeability, perceived intelligence, and perceived safety of robots,” *Int. J. Social Robotics*, vol. 1, no. 1, pp. 71–81, 2009.
- [12] C. M. Carpinella, A. B. Wyman, M. A. Perez, and S. J. Stroessner, “The Robotic Social Attributes Scale (RoSAS): Development and validation,” in *Proc. ACM/IEEE Int. Conf. Human-Robot Interaction (HRI)*, 2017, pp. 254–262.
- [13] T. Nomura, T. Suzuki, T. Kanda, and K. Kato, “Measurement of negative attitudes toward robots,” *Interaction Studies*, vol. 7, no. 3, pp. 437–454, 2006.

Appendix A: Code

Comparing Verbal and Non-Verbal Communication Styles in an LLM-Powered Accountability Robot

Omar Ankit — University of Waterloo

Note: Code highlighted in yellow indicates original contributions by the author (commits 8ae91de and f73742e). All other code is from the base mindmodel.ai/ageai.io reachy-mini-elevenlabs integration.

1. emotion_detector.py

```

1  """Emotion detection from text for triggering robot expressions.
2
3  This module analyzes agent text responses to detect emotional content
4  and map it to appropriate robot movements and expressions.
5
6  ATtribution NOTICE:
7  Emotion triggering approach inspired by reachy_mini_conversation_app
8  Original work Copyright (c) Pollen Robotics
9  Licensed under the Apache License, Version 2.0
10 Source: https://github.com/pollen-robotics/reachy\_mini\_conversation\_app
11 """
12
13 from __future__ import annotations
14
15 import logging
16 import re
17 from typing import TYPE_CHECKING
18
19 if TYPE_CHECKING:
20     from reachy_mini_elevenlabs.tools.core_tools import ToolDependencies
21
22
23 logger = logging.getLogger(__name__)
24
25
26 # Initialize emotion library
27 try:
28     from reachy_mini.motion.recorded_move import RecordedMoves
29     from reachy_mini_elevenlabs.dance_emotion_moves import (
30         DanceQueueMove,
31         EmotionQueueMove,
32         GotoQueueMove,
33     )
34     from reachy_mini.utils import create_head_pose
35
36     RECORDED_MOVES = RecordedMoves("pollen-robotics/reachy-mini-emotions-library")
37     EMOTION_AVAILABLE = True
38     logger.info("Emotion library loaded successfully")
39 except ImportError as e:
40     logger.warning(f"Emotion library not available: {e}")
41     RECORDED_MOVES = None
42     EMOTION_AVAILABLE = False
43     DanceQueueMove = None
44     EmotionQueueMove = None
45     GotoQueueMove = None
46
47 try:
48     from reachy_mini_dances_library.collection.dance import AVAILABLE_MOVES as DANCE_MOVES
49     DANCE_AVAILABLE = True
50 except ImportError:
51     DANCE_MOVES = {}
52     DANCE_AVAILABLE = False
53
54

```

```

55 EMOTION_PATTERNS = {
56     "happy": {
57         "keywords": ["happy", "glad", "great", "wonderful", "excellent", "fantastic",
58             "excited", "thrilled", "delighted", "pleased", "joy", "cheerful",
59             "awesome", "amazing", "perfect", "love", "yay", "hooray"],
60         "patterns": [r":\)", r"\U0001f60a|\U0001f604|\U0001f603|\U0001f389|\u2728"],
61         "action": "play_emotion", "action_param": "cheerful1",
62     },
63     "enthusiastic": {
64         "keywords": ["incredible", "extraordinary", "celebrate", "congratulations",
65             "victory", "triumph", "phenomenal", "outstanding"],
66         "patterns": [r"!{3,}"],
67         "action": "play_emotion", "action_param": "enthusiastic1",
68     },
69     "proud": {
70         "keywords": ["proud", "accomplished", "nailed it", "did it", "success",
71             "achieved", "well done", "good job", "bravo"],
72         "patterns": [],
73         "action": "play_emotion", "action_param": "proud2",
74     },
75     "grateful": {
76         "keywords": ["thank you", "thanks", "grateful", "appreciate", "gratitude",
77             "thankful", "much appreciated"],
78         "patterns": [],
79         "action": "play_emotion", "action_param": "grateful1",
80     },
81     "admiration": {
82         "keywords": ["admire", "impressive", "remarkable", "brilliant", "genius",
83             "magnificent", "superb", "spectacular"],
84         "patterns": [],
85         "action": "play_emotion", "action_param": "admiration1",
86     },
87     "relief": {
88         "keywords": ["relief", "relieved", "finally", "phew", "at last",
89             "thank goodness", "that's over"],
90         "patterns": [],
91         "action": "play_emotion", "action_param": "relief1",
92     },
93     "laughing": {
94         "keywords": ["haha", "hehe", "lol", "funny", "hilarious", "joke",
95             "laugh", "laughing", "comedy", "crack up"],
96         "patterns": [r"\U0001f602|\U0001f923|\U0001f606"],
97         "action": "play_emotion", "action_param": "laughing1",
98     },
99     "sad": {
100         "keywords": ["sad", "sorry", "unfortunately", "regret", "apologize",
101             "disappointed", "unhappy", "upset", "down", "blue",
102             "gloomy", "melancholy", "heartbroken"],
103         "patterns": [r":\(", r"\U0001f622|\U0001f61e|\U0001f614|\U0001f61f"],
104         "action": "play_emotion", "action_param": "sad1",
105     },
106     "discouraged": {
107         "keywords": ["discouraged", "hopeless", "despair", "give up", "pointless",
108             "no use", "what's the point", "can't do this"],
109         "patterns": [],
110         "action": "play_emotion", "action_param": "discouraged1",
111     },
112     "frustrated": {
113         "keywords": ["frustrated", "frustrating", "can't figure", "stuck",
114             "impossible", "no solution", "give up"],
115         "patterns": [],
116         "action": "play_emotion", "action_param": "frustrated1",
117     },
118     "angry": {
119         "keywords": ["angry", "mad", "furious", "rage", "outraged", "livid",
120             "infuriated", "irate"],
121         "patterns": [r"\U0001f620|\U0001f621"],
122         "action": "play_emotion", "action_param": "furious1",

```

```

123 },
124 "irritated": {
125     "keywords": ["irritated", "annoyed", "bothered", "bugged", "ugh",
126         "come on", "seriously"],
127     "patterns": [],
128     "action": "play_emotion", "action_param": "irritated1",
129 },
130 "disapproving": {
131     "keywords": ["disapprove", "careless", "disrespectful", "rude",
132         "inappropriate", "unacceptable", "shame on"],
133     "patterns": [],
134     "action": "play_emotion", "action_param": "disapproving1",
135 },
136 "reprimand": {
137     "keywords": ["stop it", "knock it off", "what's wrong with you",
138         "ridiculous", "silly", "nonsense", "foolish"],
139     "patterns": [],
140     "action": "play_emotion", "action_param": "reprimand3",
141 },
142 "thinking": {
143     "keywords": ["hmm", "let me think", "considering", "analyzing", "processing",
144         "evaluating", "pondering", "wondering", "interesting"],
145     "patterns": [r"\.{3,}", r"\U0001f914"],
146     "action": "play_emotion", "action_param": "thoughtful1",
147 },
148 "confused": {
149     "keywords": ["confused", "unclear", "not sure", "don't understand", "puzzled",
150         "perplexed", "baffled", "uncertain", "unsure", "lost"],
151     "patterns": [r"\.{2,}", r"\U0001f615|\U0001f937"],
152     "action": "play_emotion", "action_param": "confused1",
153 },
154 "surprised": {
155     "keywords": ["wow", "surprising", "unexpected", "astonishing",
156         "shocking", "no way", "boo"],
157     "patterns": [r"!{2,}", r"\U0001f62e|\U0001f632|\U0001f92f"],
158     "action": "play_emotion", "action_param": "surprised1",
159 },
160 "greeting": {
161     "keywords": ["hello", "hi", "hey", "greetings", "good morning",
162         "good afternoon", "good evening", "welcome", "howdy"],
163     "patterns": [r"\U0001f44b"],
164     "action": "play_emotion", "action_param": "welcoming1",
165 },
166 "dance_celebrate": {
167     "keywords": ["dance", "dancing", "party", "celebrate", "groove",
168         "music", "bust a move", "boogie"],
169     "patterns": [r"\U0001f483|\U0001f57a|\U0001f3b6|\U0001f3b5"],
170     "action": "dance", "action_param": "groovy_sway_and_roll",
171 },
172 }
173
174
175 # Personality weight profiles for DAMN-inspired weighted arbitration
176 PERSONALITY_WEIGHTS = {
177     "sarcastic": {
178         "reprimand": 1.5, "irritated": 1.3, "disapproving": 1.3,
179         "laughing": 1.3, "surprised": 1.2, "impatient": 1.2,
180         "happy": 0.5, "enthusiastic": 0.4, "proud": 0.5,
181         "dance_celebrate": 0.3, "admiration": 0.4,
182     },
183     "motivational": {
184         "enthusiastic": 1.5, "proud": 1.4, "happy": 1.3,
185         "dance_celebrate": 1.5, "admiration": 1.3, "success": 1.3,
186         "grateful": 1.2,
187         "reprimand": 0.3, "disapproving": 0.3, "angry": 0.3,
188         "irritated": 0.4, "sad": 0.5, "frustrated": 0.4,
189     },
190     "neutral": {

```

```

191     "happy": 0.1, "enthusiastic": 0.1, "proud": 0.1,
192     "sad": 0.1, "angry": 0.1, "irritated": 0.1,
193     "reprimand": 0.1, "disapproving": 0.1, "laughing": 0.1,
194     "dance_celebrate": 0.0, "dance_nod": 0.0,
195     "surprised": 0.1, "scared": 0.1, "admiration": 0.1,
196     "greeting": 0.8,
197 },
198 }
199
200
201 class EmotionDetector:
202     """Detects emotions from text and triggers robot expressions.
203
204     Implements a weighted arbitration architecture inspired by Rosenblatt
205     and Payton's DAMN (1989). Detected behaviours are grouped into
206     independent channels (emotion, dance, head movement) and the best
207     candidate in each channel is selected via personality-weighted
208     confidence scores.
209     """
210
211     def __init__(self, deps, enabled=True, min_confidence=0.3,
212                  cooldown_seconds=3.0, personality=None):
213         self.deps = deps
214         self.enabled = enabled
215         self.min_confidence = min_confidence
216         self.cooldown_seconds = cooldown_seconds
217         self.personality = personality
218         self._last_action_time = 0.0
219         self._last_action_time_by_category = {}
220
221     def analyze_and_act(self, text):
222         """Analyze text and trigger actions from each behaviour channel."""
223         if not self.enabled:
224             return {"detected_emotions": [], "actions_triggered": [],
225                     "action_triggered": False}
226         import time
227         current_time = time.time()
228
229         all_scores = self._score_all_patterns(text)
230         weighted_scores = self._apply_personality_weights(all_scores)
231         category_winners = self._select_per_category(weighted_scores)
232
233         actions_triggered = []
234         for category, (emotion_name, confidence) in category_winners.items():
235             if confidence < self.min_confidence:
236                 continue
237             last_time = self._last_action_time_by_category.get(category, 0.0)
238             if current_time - last_time < self.cooldown_seconds:
239                 continue
240             if self._trigger_action(emotion_name):
241                 self._last_action_time_by_category[category] = current_time
242                 self._last_action_time = current_time
243                 actions_triggered.append({
244                     "emotion": emotion_name, "category": category,
245                     "confidence": confidence,
246                     "action_type": EMOTION_PATTERNS[emotion_name]["action"],
247                 })
248
249         first = actions_triggered[0] if actions_triggered else None
250         return {
251             "detected_emotion": first["emotion"] if first else None,
252             "confidence": first["confidence"] if first else 0.0,
253             "action_triggered": len(actions_triggered) > 0,
254             "action_type": first["action_type"] if first else None,
255             "detected_emotions": [a["emotion"] for a in actions_triggered],
256             "actions_triggered": actions_triggered,
257         }
258

```



```

259 def _score_all_patterns(self, text):
260     text_lower = text.lower()
261     results = []
262     for emotion_name, emotion_data in EMOTION_PATTERNS.items():
263         confidence = 0.0
264         matches = 0
265         for keyword in emotion_data["keywords"]:
266             if keyword in text_lower:
267                 matches += 1
268                 confidence += len(keyword) / 20.0
269         for pattern in emotion_data["patterns"]:
270             if re.search(pattern, text):
271                 matches += 1
272                 confidence += 0.5
273         if matches > 0:
274             confidence = min(confidence, 1.0)
275             results.append((emotion_name, confidence))
276     return results
277
278 def _apply_personality_weights(self, scores):
279     if not self.personality or self.personality not in PERSONALITY_WEIGHTS:
280         return scores
281     weights = PERSONALITY_WEIGHTS[self.personality]
282     return [(name, conf * weights.get(name, 1.0)) for name, conf in scores]
283
284 @staticmethod
285 def _select_per_category(scores):
286     buckets = {}
287     for emotion_name, confidence in scores:
288         category = EMOTION_PATTERNS[emotion_name]["action"]
289         buckets.setdefault(category, []).append((emotion_name, confidence))
290     winners = {}
291     for category, candidates in buckets.items():
292         best = max(candidates, key=lambda x: x[1])
293         winners[category] = best
294     return winners
295
296 def _trigger_action(self, emotion):
297     if not EMOTION_AVAILABLE:
298         return False
299     if emotion not in EMOTION_PATTERNS:
300         return False
301     emotion_data = EMOTION_PATTERNS[emotion]
302     action_type = emotion_data["action"]
303     action_param = emotion_data["action_param"]
304     try:
305         if action_type == "play_emotion":
306             emotion_names = RECORDED_MOVES.list_moves()
307             if action_param not in emotion_names:
308                 return False
309             emotion_move = EmotionQueueMove(action_param, RECORDED_MOVES)
310             self.deps.movement_manager.queue_move(emotion_move)
311             return True
312         elif action_type == "dance":
313             if not DANCE_AVAILABLE or action_param not in DANCE_MOVES:
314                 return False
315             dance_move = DanceQueueMove(action_param)
316             self.deps.movement_manager.queue_move(dance_move)
317             return True
318         return False
319     except Exception as e:
320         logger.error(f"Error triggering action for emotion {emotion}: {e}")
321         return False

```

2. config.py

```

1  """Configuration management for ElevenLabs integration.
2
3  This module handles loading and validating configuration from environment
4  variables and '.env' files using python-dotenv.
5
6  Get your ElevenLabs agent at: https://try.elevenlabs.io/reachy-mini-agents
7  """
8
9  from __future__ import annotations
10 import os
11 import logging
12
13 from dotenv import load_dotenv
14
15
16 logger = logging.getLogger(__name__)
17
18
19 class Config:
20     """Configuration for the ElevenLabs app.
21
22     Loads configuration from environment variables and '.env' files.
23     The ELEVENLABS_AGENT_ID is required for all operations.
24     The ELEVENLABS_API_KEY is optional but required for private agents.
25     """
26
27     def __init__(self) -> None:
28         """Initialize configuration by loading from environment."""
29         # Load from .env file if present (does not override existing env vars)
30         load_dotenv()
31
32         # Required: Agent ID from ElevenLabs dashboard
33         self.ELEVENLABS_AGENT_ID: str | None = os.getenv("ELEVENLABS_AGENT_ID") or None
34
35         # Optional: API key (required for private agents)
36         self.ELEVENLABS_API_KEY: str | None = os.getenv("ELEVENLABS_API_KEY") or None
37
38         # Emotion detection settings
39         self.ENABLE_EMOTION_DETECTION: bool = os.getenv("ENABLE_EMOTION_DETECTION", "true").lower() in ("
40 true", "1", "yes")
41         self.EMOTION_CONFIDENCE_THRESHOLD: float = float(os.getenv("EMOTION_CONFIDENCE_THRESHOLD", "0.3"))
42         self.EMOTION_COOLDOWN_SECONDS: float = float(os.getenv("EMOTION_COOLDOWN_SECONDS", "3.0"))
43
44         # Idle emotion settings
45         self.ENABLE_IDLE_EMOTIONS: bool = os.getenv("ENABLE_IDLE_EMOTIONS", "true").lower() in ("true", "1
46 ", "yes")
47         self.IDLE_EMOTION_MIN_DELAY: float = float(os.getenv("IDLE_EMOTION_MIN_DELAY", "3.0"))
48         self.IDLE_EMOTION_MAX_DELAY: float = float(os.getenv("IDLE_EMOTION_MAX_DELAY", "8.0"))
49
50
51         # Personality for weighted emotion arbitration
52         # Values: "sarcastic", "motivational", "neutral", "drill_sergeant", or empty for no weighting
53         self.PERSONALITY: str | None = os.getenv("PERSONALITY") or None
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

51
52 def validate(self, log_errors: bool = True) -> list[str]:
53     """Return list of missing required configuration."""
54     errors: list[str] = []
55     if not self.ELEVENLABS_AGENT_ID:
56         errors.append("ELEVENLABS_AGENT_ID")
57         if log_errors:
58             logger.error("Missing required configuration: ELEVENLABS_AGENT_ID")
59     return errors
60
61
62 def log_missing_api_key_warning(self) -> None:
63     """Log a warning if API key is missing."""
64     if not self.ELEVENLABS_API_KEY:
65         logger.error(

```

```

65         "Missing configuration: ELEVENLABS_API_KEY. "
66         "This is required for private agents."
67     )
68
69     def reload(self) -> None:
70         """Reload configuration from environment."""
71         load_dotenv(override=True)
72         self.ELEVENLABS_AGENT_ID = os.getenv("ELEVENLABS_AGENT_ID") or None
73         self.ELEVENLABS_API_KEY = os.getenv("ELEVENLABS_API_KEY") or None
74         self.ENABLE_EMOTION_DETECTION = os.getenv("ENABLE_EMOTION_DETECTION", "true").lower() in ("true",
75 "1", "yes")
76         self.EMOTION_CONFIDENCE_THRESHOLD = float(os.getenv("EMOTION_CONFIDENCE_THRESHOLD", "0.3"))
77         self.EMOTION_COOLDOWN_SECONDS = float(os.getenv("EMOTION_COOLDOWN_SECONDS", "3.0"))
78         self.ENABLE_IDLE_EMOTIONS = os.getenv("ENABLE_IDLE_EMOTIONS", "true").lower() in ("true", "1", "
79 yes")
80         self.IDLE_EMOTION_MIN_DELAY = float(os.getenv("IDLE_EMOTION_MIN_DELAY", "3.0"))
81         self.IDLE_EMOTION_MAX_DELAY = float(os.getenv("IDLE_EMOTION_MAX_DELAY", "8.0"))
82
83         self.PERSONALITY = os.getenv("PERSONALITY") or None
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

3. elevenlabs_handler.py

```

1  """ElevenLabs conversation handler for Reachy Mini."""
2  from __future__ import annotations
3  import logging, time
4  from typing import TYPE_CHECKING
5  from elevenlabs import ElevenLabs
6  from elevenlabs.conversational_ai.conversation import ClientTools, Conversation
7  from reachy_mini_elevenlabs.audio_interface import ReachyAudioInterface
8  from reachy_mini_elevenlabs.config import config
9  from reachy_mini_elevenlabs.emotion_detector import EmotionDetector
10 from reachy_mini_elevenlabs.tools.core_tools import ToolDependencies, register_tools
11 logger = logging.getLogger(__name__)
12 DEFAULT_MAX_RECONNECT_ATTEMPTS = 5
13 DEFAULT_INITIAL_BACKOFF_SECONDS = 1.0
14 DEFAULT_MAX_BACKOFF_SECONDS = 30.0
15
16 class ElevenLabsHandler:
17     """Manages ElevenLabs Conversation and coordinates components."""
18
19     def __init__(self, deps, instance_path=None,
20                 max_reconnect_attempts=DEFAULT_MAX_RECONNECT_ATTEMPTS,
21                 initial_backoff=DEFAULT_INITIAL_BACKOFF_SECONDS,
22                 max_backoff=DEFAULT_MAX_BACKOFF_SECONDS,
23                 backoff_multiplier=2.0, enable_emotion_detection=True):
24         self.deps = deps
25         self.instance_path = instance_path
26         self.client = None
27         self.conversation = None
28         self.audio_interface = None
29         self.emotion_detector = EmotionDetector(
30             deps=deps,
31             enabled=enable_emotion_detection and config.ENABLE_EMOTION_DETECTION,
32             min_confidence=config.EMOTION_CONFIDENCE_THRESHOLD,
33             cooldown_seconds=config.EMOTION_COOLDOWN_SECONDS,
34
35             personality=config.PERSONALITY, # <<< AUTHOR CONTRIBUTION
36         )

```

```

36     self.max_reconnect_attempts = max_reconnect_attempts
37     self.initial_backoff = initial_backoff
38     self.max_backoff = max_backoff
39     self.backoff_multiplier = backoff_multiplier
40     self._should_restart = True
41     self._stop_requested = False
42
43     def start(self):
44         missing = config.validate()
45         if missing:
46             raise ValueError(f"Missing required configuration: {', '.join(missing)}")
47         logger.info(
48             f"Starting ElevenLabs conversation handler "
49
50             f"(personality={config.PERSONALITY or 'none'})" # <<< AUTHOR CONTRIBUTION
51         )
52
53     self._stop_requested = False
54     self.client = ElevenLabs(api_key=config.ELEVENLABS_API_KEY)
55     self.audio_interface = ReachyAudioInterface(
56         robot=self.deps.reachy_mini, head_wobbler=self.deps.head_wobbler)
57     client_tools = self._create_client_tools()
58     api_key = config.ELEVENLABS_API_KEY
59     requires_auth = bool(api_key and api_key.strip())
60     self._create_conversation(client_tools, requires_auth)
61     self._start_session_with_retry()
62
63     def stop(self):
64         self._stop_requested = True
65         self._should_restart = False
66         if self.conversation:
67             try: self.conversation.end_session()
68             except Exception as e: logger.warning(f"Error ending session: {e}")
69
70     def wait_for_end(self):
71         if not self.conversation: return ""
72         while True:
73             try:
74                 conversation_id = self.conversation.wait_for_session_end()
75                 if self._should_restart and not self._stop_requested:
76                     if self._attempt_session_restart(): continue
77                 return conversation_id
78             except Exception:
79                 if self._should_restart and not self._stop_requested:
80                     if self._attempt_session_restart(): continue
81                 return ""
82
83     def _create_client_tools(self):
84         client_tools = ClientTools()
85         register_tools(client_tools, self.deps)
86         return client_tools
87
88     def _on_agent_response(self, response):
89         logger.info(f"Agent: {response}")
90         emotion_result = self.emotion_detector.analyze_and_act(response)
91         if emotion_result["action_triggered"]:
92             logger.debug(f"Emotion: {emotion_result['detected_emotion']} "
93                         f"(confidence: {emotion_result['confidence']:.2f})")
94
95     def _on_user_transcript(self, transcript):
96         logger.info(f"User: {transcript}")
97
98     def _create_conversation(self, client_tools, requires_auth):
99         self.conversation = Conversation(
100             client=self.client, agent_id=config.ELEVENLABS_AGENT_ID,
            requires_auth=requires_auth, audio_interface=self.audio_interface,
            client_tools=client_tools,

```

```

101         callback_agent_response=self._on_agent_response,
102         callback_user_transcript=self._on_user_transcript)
103
104     def _start_session_with_retry(self):
105         backoff = self.initial_backoff
106         for attempt in range(self.max_reconnect_attempts):
107             try:
108                 self.conversation.start_session()
109                 logger.info("Session started")
110                 return
111             except Exception as e:
112                 logger.error(f"Connection failure (attempt {attempt+1}): {e}")
113                 if attempt < self.max_reconnect_attempts - 1:
114                     time.sleep(backoff)
115                     backoff = min(backoff * self.backoff_multiplier, self.max_backoff)
116                 raise ConnectionError("Failed to start after max attempts")
117
118     def _attempt_session_restart(self):
119         backoff = self.initial_backoff
120         for attempt in range(self.max_reconnect_attempts):
121             try:
122                 client_tools = self._create_client_tools()
123                 api_key = config.ELEVENLABS_API_KEY
124                 self._create_conversation(client_tools, bool(api_key and api_key.strip()))
125                 self.conversation.start_session()
126                 return True
127             except Exception:
128                 if attempt < self.max_reconnect_attempts - 1:
129                     time.sleep(backoff)
130                     backoff = min(backoff * self.backoff_multiplier, self.max_backoff)
131         return False

```

4. idle_emotions.py

```

1  """Idle emotion system for continuous subtle expressions.
2
3  ATtribution NOTICE:
4  Idle emotion concept inspired by reachy_mini_conversation_app breathing system
5  Original work Copyright (c) Pollen Robotics, Apache License 2.0
6  """
7  from __future__ import annotations
8  import logging, random, time, threading
9  from typing import TYPE_CHECKING
10 if TYPE_CHECKING:
11     from reachy_mini_elevenlabs.tools.core_tools import ToolDependencies
12 logger = logging.getLogger(__name__)
13
14 try:
15     from reachy_mini.motion.recorded_move import RecordedMoves
16     from reachy_mini.elevenlabs.dance_emotion_moves import EmotionQueueMove
17     RECORDED_MOVES = RecordedMoves("pollen-robotics/reachy-mini-emotions-library")
18     EMOTION_AVAILABLE = True
19 except ImportError as e:
20     RECORDED_MOVES = None
21     EMOTION_AVAILABLE = False
22     EmotionQueueMove = None
23
24 IDLE_EMOTIONS = [
25     "understanding1", "understanding2", "indifferent1", "curious1",
26     "attentive1", "attentive2", "shy1", "serenity1",
27     "proud1", "proud2", "proud3", "thoughtful1", "thoughtful2",
28 ]
29
30 class IdleEmotionManager:
31     def __init__(self, deps, enabled=True, min_delay=3.0, max_delay=8.0):
32         self.deps = deps

```

```

33     self.enabled = enabled
34     self.min_delay = min_delay
35     self.max_delay = max_delay
36     self._stop_event = threading.Event()
37     self._thread = None
38     self._last_idle_time = time.time()
39     if EMOTION_AVAILABLE:
40         available = RECORDED_MOVES.list_moves()
41         self.available_idle_emotions = [e for e in IDLE_EMOTIONS if e in available]
42         if not self.available_idle_emotions: self.enabled = False
43     else:
44         self.enabled = False
45         self.available_idle_emotions = []
46
47     def start(self):
48         if not self.enabled or not EMOTION_AVAILABLE: return
49         self._stop_event.clear()
50         self._thread = threading.Thread(target=self._idle_loop, daemon=True)
51         self._thread.start()
52
53     def stop(self):
54         if self._thread is None: return
55         self._stop_event.set()
56         if self._thread.is_alive(): self._thread.join(timeout=2.0)
57         self._thread = None
58
59     def _idle_loop(self):
60         while not self._stop_event.is_set():
61             try:
62                 if self._should_queue_idle_emotion():
63                     self._queue_random_idle_emotion()
64                 time.sleep(0.5)
65             except Exception as e:
66                 logger.error(f"Idle emotion error: {e}")
67                 time.sleep(1.0)
68
69     def _should_queue_idle_emotion(self):
70         """Check if robot is truly idle before queueing."""
71         if not self.enabled: return False
72         current_time = time.time()
73         if current_time - self._last_idle_time < random.uniform(self.min_delay, self.max_delay):
74             return False
75         # Fixed idle check: verify no move playing AND no moves queued
76         try:
77             mm = self.deps.movement_manager
78             if hasattr(mm, 'state') and mm.state.current_move is not None:
79                 from reachy_mini_elevenlabs.moves import BreathingMove
80                 if not isinstance(mm.state.current_move, BreathingMove):
81                     return False
82             if hasattr(mm, 'move_queue') and len(mm.move_queue) > 0:
83                 return False
84         except Exception:
85             pass
86         return True
87
88     def _queue_random_idle_emotion(self):
89         if not self.available_idle_emotions: return
90         try:
91             emotion_name = random.choice(self.available_idle_emotions)
92             emotion_move = EmotionQueueMove(emotion_name, RECORDED_MOVES)
93             self.deps.movement_manager.queue_move(emotion_move)
94             self._last_idle_time = time.time()
95         except Exception as e:
96             logger.error(f"Failed to queue idle emotion: {e}")

```


5. dance_emotion_moves.py

```

1  """Dance and emotion moves for the movement queue system.
2
3  ATtribution NOTICE:
4  Derived from reachy_mini_conversation_app/dance_emotion_moves.py
5  Original work Copyright (c) Pollen Robotics, Apache License 2.0
6  """
7
8  from __future__ import annotations
9  import logging
10 from typing import Tuple
11 import numpy as np
12 from numpy.typing import NDArray
13 from reachy_mini.motion.move import Move
14 from reachy_mini.motion.recorded_move import RecordedMoves
15 from reachy_mini_dances_library.dance_move import DanceMove
16
17 logger = logging.getLogger(__name__)
18
19 class DanceQueueMove(Move):
20     def __init__(self, move_name):
21         self.dance_move = DanceMove(move_name)
22         self.move_name = move_name
23
24     @property
25     def duration(self):
26         return float(self.dance_move.duration)
27
28     def evaluate(self, t):
29         try:
30             head_pose, antennas, body_yaw = self.dance_move.evaluate(t)
31             if isinstance(antennas, tuple):
32                 antennas = np.array([antennas[0], antennas[1]])
33             return (head_pose, antennas, body_yaw)
34         except Exception as e:
35             logger.error(f"Error evaluating dance '{self.move_name}': {e}")
36             from reachy_mini.utils import create_head_pose
37             return (create_head_pose(0,0,0,0,0, degrees=True), np.array([0.0,0.0]), 0.0)
38
39 class EmotionQueueMove(Move):
40     def __init__(self, emotion_name, recorded_moves):
41         self.emotion_move = recorded_moves.get(emotion_name)
42         self.emotion_name = emotion_name
43
44     @property
45     def duration(self):
46         return float(self.emotion_move.duration)
47
48     def evaluate(self, t):
49         try:
50             head_pose, antennas, body_yaw = self.emotion_move.evaluate(t)
51             if isinstance(antennas, tuple):
52                 antennas = np.array([antennas[0], antennas[1]])
53             return (head_pose, antennas, body_yaw)
54         except Exception as e:
55             logger.error(f"Error evaluating emotion '{self.emotion_name}': {e}")
56             from reachy_mini.utils import create_head_pose
57             return (create_head_pose(0,0,0,0,0, degrees=True), np.array([0.0,0.0]), 0.0)
58
59 class GotoQueueMove(Move):
60     def __init__(self, target_head_pose, start_head_pose=None,
61                  target_antennas=(0,0), start_antennas=None,
62                  target_body_yaw=0, start_body_yaw=None, duration=1.0):
63         self._duration = duration
64         self.target_head_pose = target_head_pose
65         self.start_head_pose = start_head_pose
66         self.target_antennas = target_antennas

```

```

67     self.start_antennas = start_antennas or (0, 0)
68     self.target_body_yaw = target_body_yaw
69     self.start_body_yaw = start_body_yaw or 0
70
71     @property
72     def duration(self):
73         return self._duration
74
75     def evaluate(self, t):
76         from reachy_mini.utils import create_head_pose
77         from reachy_mini.utils.interpolation import linear_pose_interpolation
78         t_clamped = max(0, min(1, t / self.duration))
79         start = self.start_head_pose if self.start_head_pose is not None \
80             else create_head_pose(0,0,0,0,0,0, degrees=True)
81         head_pose = linear_pose_interpolation(start, self.target_head_pose, t_clamped)
82         antennas = np.array([
83             self.start_antennas[0] + (self.target_antennas[0] - self.start_antennas[0]) * t_clamped,
84             self.start_antennas[1] + (self.target_antennas[1] - self.start_antennas[1]) * t_clamped,
85         ], dtype=np.float64)
86         body_yaw = self.start_body_yaw + (self.target_body_yaw - self.start_body_yaw) * t_clamped
87         return (head_pose, antennas, body_yaw)

```

6. moves.py

```

1  """Movement system: sequential primary moves + additive secondary moves.
2
3  ATTRIBUTION NOTICE:
4  Derived from reachy_mini_conversation_app/moves.py
5  Original work Copyright (c) Pollen Robotics, Apache License 2.0
6
7  Design: Primary moves (emotions, dances, goto, breathing) are mutually exclusive
8  and run sequentially. Secondary moves (speech sway) are additive offsets applied
9  on top of the current primary pose. Single control point: ReachyMini.set_target.
10 The control loop runs near 100 Hz.
11 """
12 from __future__ import annotations
13 import time, logging, threading
14 from queue import Empty, Queue
15 from typing import Any, Dict, Tuple
16 from collections import deque
17 from dataclasses import dataclass
18 import numpy as np
19 from numpy.typing import NDArray
20 from reachy_mini import ReachyMini
21 from reachy_mini.utils import create_head_pose
22 from reachy_mini.motion.move import Move
23 from reachy_mini.utils.interpolation import compose_world_offset, linear_pose_interpolation
24 logger = logging.getLogger(__name__)
25 CONTROL_LOOP_FREQUENCY_HZ = 100.0
26 FullBodyPose = Tuple[NDArray[np.float32], Tuple[float, float], float]
27
28 class BreathingMove(Move):
29     """Breathing with interpolation to neutral then continuous patterns."""
30     def __init__(self, interpolation_start_pose, interpolation_start_antennas,
31                 interpolation_duration=1.0):
32         self.interpolation_start_pose = interpolation_start_pose
33         self.interpolation_start_antennas = np.array(interpolation_start_antennas)
34         self.interpolation_duration = interpolation_duration
35         self.neutral_head_pose = create_head_pose(0,0,0,0,0,0, degrees=True)
36         self.neutral_antennas = np.array([0.0, 0.0])
37         self.breathing_z_amplitude = 0.005
38         self.breathing_frequency = 0.1
39         self.antenna_sway_amplitude = np.deg2rad(15)
40         self.antenna_frequency = 0.5
41
42     @property

```

```

43     def duration(self): return float("inf")
44
45     def evaluate(self, t):
46         if t < self.interpolation_duration:
47             it = t / self.interpolation_duration
48             head_pose = linear_pose_interpolation(
49                 self.interpolation_start_pose, self.neutral_head_pose, it)
50             antennas = ((1-it)*self.interpolation_start_antennas +
51                 it*self.neutral_antennas).astype(np.float64)
52         else:
53             bt = t - self.interpolation_duration
54             z = self.breathing_z_amplitude * np.sin(2*np.pi*self.breathing_frequency*bt)
55             head_pose = create_head_pose(x=0,y=0,z=z,roll=0,pitch=0,yaw=0,degrees=True,mm=False)
56             sway = self.antenna_sway_amplitude * np.sin(2*np.pi*self.antenna_frequency*bt)
57             antennas = np.array([sway, -sway], dtype=np.float64)
58         return (head_pose, antennas, 0.0)
59
60     def combine_full_body(primary, secondary):
61         ph, pa, pb = primary; sh, sa, sb = secondary
62         ch = compose_world_offset(ph, sh, reorthonormalize=True)
63         ca = (pa[0]+sa[0], pa[1]+sa[1]); cb = pb + sb
64         return (ch, ca, cb)
65
66     def clone_full_body_pose(pose):
67         h, a, b = pose
68         return (h.copy(), (float(a[0]), float(a[1])), float(b))
69
70 @dataclass
71 class MovementState:
72     current_move: Move | None = None
73     move_start_time: float | None = None
74     last_activity_time: float = 0.0
75     speech_offsets: Tuple[float,float,float,float,float,float] = (0,0,0,0,0,0)
76     face_tracking_offsets: Tuple[float,float,float,float,float,float] = (0,0,0,0,0,0)
77     last_primary_pose: FullBodyPose | None = None
78     def update_activity(self): self.last_activity_time = time.monotonic()
79
80 class MovementManager:
81     """Coordinate sequential moves, additive offsets, and robot output at 100 Hz."""
82     def __init__(self, current_robot, camera_worker=None):
83         self.current_robot = current_robot
84         self.camera_worker = camera_worker
85         self._now = time.monotonic
86         self.state = MovementState()
87         self.state.last_activity_time = self._now()
88         neutral = create_head_pose(0,0,0,0,0,0, degrees=True)
89         self.state.last_primary_pose = (neutral, (0.0, 0.0), 0.0)
90         self.move_queue = deque()
91         self.idle_inactivity_delay = 0.3
92         self.target_period = 1.0 / CONTROL_LOOP_FREQUENCY_HZ
93         self._stop_event = threading.Event()
94         self._thread = None
95         self._is_listening = False
96         self._last_commanded_pose = clone_full_body_pose(self.state.last_primary_pose)
97         self._listening_antennas = self._last_commanded_pose[1]
98         self._antenna_unfreeze_blend = 1.0
99         self._antenna_blend_duration = 0.4
100         self._last_listening_blend_time = self._now()
101         self._breathing_active = False
102         self._command_queue = Queue()
103         self._speech_offsets_lock = threading.Lock()
104         self._pending_speech_offsets = (0,0,0,0,0,0)
105         self._speech_offsets_dirty = False
106         self._shared_state_lock = threading.Lock()
107
108     def queue_move(self, move):
109         self._command_queue.put(("queue_move", move))
110

```

```

111 def clear_move_queue(self):
112     self._command_queue.put(("clear_queue", None))
113
114 def set_speech_offsets(self, offsets):
115     with self._speech_offsets_lock:
116         self._pending_speech_offsets = offsets
117         self._speech_offsets_dirty = True
118
119 def set_listening(self, listening):
120     self._command_queue.put(("set_listening", listening))
121
122 def start(self):
123     self._stop_event.clear()
124     self._thread = threading.Thread(target=self.working_loop, daemon=True)
125     self._thread.start()
126
127 def stop(self):
128     self.clear_move_queue()
129     self._stop_event.set()
130     if self._thread: self._thread.join(); self._thread = None
131     try:
132         neutral = create_head_pose(0,0,0,0,0,0, degrees=True)
133         self.current_robot.goto_target(head=neutral, antennas=[0,0],
134                                     duration=2.0, body_yaw=0)
135     except Exception as e:
136         logger.error(f"Failed to reset: {e}")
137
138 def working_loop(self):
139     """100 Hz control loop: poll commands, sample move, fuse offsets, set_target."""
140     while not self._stop_event.is_set():
141         loop_start = self._now()
142         # Poll commands
143         with self._speech_offsets_lock:
144             if self._speech_offsets_dirty:
145                 self.state.speech_offsets = self._pending_speech_offsets
146                 self._speech_offsets_dirty = False
147         while True:
148             try:
149                 cmd, payload = self._command_queue.get_nowait()
150                 if cmd == "queue_move":
151                     self.move_queue.append(payload)
152                     self.state.update_activity()
153                 elif cmd == "clear_queue":
154                     self.move_queue.clear()
155                     self.state.current_move = None; self._breathing_active = False
156                 elif cmd == "set_listening":
157                     self._is_listening = bool(payload)
158             except Empty: break
159         # Manage move queue
160         if self.state.current_move is None or (
161             self.state.move_start_time is not None and
162             loop_start - self.state.move_start_time >= self.state.current_move.duration):
163             self.state.current_move = None; self.state.move_start_time = None
164         if self.move_queue:
165             self.state.current_move = self.move_queue.popleft()
166             self.state.move_start_time = loop_start
167             self._breathing_active = isinstance(self.state.current_move, BreathingMove)
168         # Breathing when idle
169         if (self.state.current_move is None and not self.move_queue
170             and not self._is_listening and not self._breathing_active):
171             if loop_start - self.state.last_activity_time >= self.idle_inactivity_delay:
172                 try:
173                     _, ants = self.current_robot.get_current_joint_positions()
174                     hp = self.current_robot.get_current_head_pose()
175                     self._breathing_active = True; self.state.update_activity()
176                     self.move_queue.append(BreathingMove(hp, ants, 1.0))
177                 except: self._breathing_active = False
178             if isinstance(self.state.current_move, BreathingMove) and self.move_queue:

```

```

179         self.state.current_move = None; self._breathing_active = False
180         # Get primary pose
181         if self.state.current_move and self.state.move_start_time:
182             mt = loop_start - self.state.move_start_time
183             h, a, b = self.state.current_move.evaluate(mt)
184             if h is None: h = create_head_pose(0,0,0,0,0,degrees=True)
185             if a is None: a = np.array([0.0,0.0])
186             if b is None: b = 0.0
187             primary = (h.copy(), (float(a[0]),float(a[1])), float(b))
188             self.state.last_primary_pose = clone_full_body_pose(primary)
189         elif self.state.last_primary_pose:
190             primary = clone_full_body_pose(self.state.last_primary_pose)
191         else:
192             neutral = create_head_pose(0,0,0,0,0,degrees=True)
193             primary = (neutral, (0.0, 0.0), 0.0)
194         # Secondary offsets
195         so = self.state.speech_offsets
196         sec_head = create_head_pose(x=so[0],y=so[1],z=so[2],
197                                   roll=so[3],pitch=so[4],yaw=so[5],degrees=False,mm=False)
198         secondary = (sec_head, (0.0, 0.0), 0.0)
199         head, antennas, body_yaw = combine_full_body(primary, secondary)
200         # Set target
201         try:
202             self.current_robot.set_target(head=head, antennas=antennas, body_yaw=body_yaw)
203             self.last_commanded_pose = clone_full_body_pose((head, antennas, body_yaw))
204         except Exception as e: pass
205         # Sleep to maintain frequency
206         elapsed = self._now() - loop_start
207         sleep_t = max(0, self.target_period - elapsed)
208         if sleep_t > 0: time.sleep(sleep_t)

```

7. main.py

```

1  """Entrypoint for the Reachy Mini ElevenLabs conversation app."""
2  from __future__ import annotations
3  import sys, time, asyncio, argparse, logging, threading
4  from reachy_mini import ReachyMini, ReachyMiniApp
5  from reachy_mini_elevenlabs.tools.core_tools import ToolDependencies
6  from reachy_mini_elevenlabs.audio.head_wobbler import HeadWobbler
7  from reachy_mini_elevenlabs.elevenlabs_handler import ElevenLabsHandler
8  from reachy_mini_elevenlabs.utils import parse_args, setup_logger, log_connection_troubleshooting
9  logger = logging.getLogger(__name__)
10
11 def main():
12     args, _ = parse_args()
13     run(args)
14
15 def run(args, robot=None, app_stop_event=None, settings_app=None, instance_path=None):
16     from reachy_mini_elevenlabs.moves import MovementManager
17     from reachy_mini_elevenlabs.camera_worker import CameraWorker
18     from reachy_mini_elevenlabs.config import config
19     from pathlib import Path
20     app_logger = setup_logger(args.debug)
21
22     if instance_path:
23         try:
24             from dotenv import load_dotenv
25             env_path = Path(instance_path) / ".env"
26             if env_path.exists():
27                 load_dotenv(dotenv_path=str(env_path), override=True)
28                 config.reload()
29         except Exception: pass
30
31     if settings_app is not None:
32         try:
33             from reachy_mini_elevenlabs.settings_ui import mount_settings_routes

```

```

34     mount_settings_routes(settings_app, instance_path)
35     except Exception as e:
36         app_logger.warning(f"Failed to mount settings UI: {e}")
37
38     missing = config.validate()
39     if missing:
40         if settings_app is not None:
41             try:
42                 while not config.ELEVENLABS_AGENT_ID:
43                     time.sleep(0.5); config.reload()
44             except KeyboardInterrupt: return
45         else:
46             app_logger.error(f"Missing: {'', '.join(missing)}")
47             sys.exit(1)
48
49     if robot is None:
50         try:
51             kwargs = {}
52             if args.robot_name: kwargs["robot_name"] = args.robot_name
53             if args.no_camera: kwargs["media_backend"] = "gststreamer_no_video"
54             robot = ReachyMini(**kwargs)
55         except Exception as e:
56             app_logger.error(f"Robot init failed: {e}"); sys.exit(1)
57
58     camera_worker = CameraWorker(robot) if not args.no_camera else None
59     movement_manager = MovementManager(current_robot=robot, camera_worker=camera_worker)
60     head_wobbler = HeadWobbler(set_speech_offsets=movement_manager.set_speech_offsets)
61     deps = ToolDependencies(reachy_mini=robot, movement_manager=movement_manager,
62                           camera_worker=camera_worker, head_wobbler=head_wobbler)
63
64     from reachy_mini_elevenlabs.idle_emotions import IdleEmotionManager
65     idle_mgr = IdleEmotionManager(deps=deps, enabled=config.ENABLE_IDLE_EMOTIONS,
66                                  min_delay=config.IDLE_EMOTION_MIN_DELAY, max_delay=config.IDLE_EMOTION_MAX_DELAY)
67     handler = ElevenLabsHandler(deps, instance_path=instance_path)
68
69     movement_manager.start(); head_wobbler.start(); idle_mgr.start()
70     if camera_worker: camera_worker.start()
71
72     if app_stop_event:
73         def poll():
74             app_stop_event.wait(); handler.stop()
75             threading.Thread(target=poll, daemon=True).start()
76
77     try:
78         handler.start()
79         conversation_id = handler.wait_for_end()
80     except KeyboardInterrupt: handler.stop()
81     except Exception: handler.stop()
82     finally:
83         movement_manager.stop(); head_wobbler.stop(); idle_mgr.stop()
84         if camera_worker: camera_worker.stop()
85         try: robot.media.close()
86         except: pass
87         robot.client.disconnect(); time.sleep(1)
88
89     class ReachyMiniElevenlabs(ReachyMiniApp):
90         description = "ElevenLabs Conversational AI for Reachy Mini"
91         custom_app_url = "http://0.0.0.0:7861/"
92         dont_start_webserver = False
93         def run(self, reachy_mini, stop_event):
94             loop = asyncio.new_event_loop(); asyncio.set_event_loop(loop)
95             args, _ = parse_args()
96             run(args, robot=reachy_mini, app_stop_event=stop_event,
97                 settings_app=self.settings_app,
98                 instance_path=self._get_instance_path().parent)

```

8. audio_interface.py

```

1  """Custom audio interface for Reachy Mini robot.
2
3  Routes audio through the robot's mic/speaker while integrating
4  with the HeadWobbler for lip-sync animation.
5  ElevenLabs format: 16-bit PCM mono at 16kHz
6  """
7  from __future__ import annotations
8  import base64, logging, threading, time
9  from collections.abc import Callable
10 from typing import TYPE_CHECKING
11 import numpy as np
12 from elevenlabs.conversational_ai.conversation import AudioInterface
13 from reachy_mini_elevenlabs.audio.utils import audio_to_float32
14 if TYPE_CHECKING:
15     from reachy_mini import ReachyMini
16     from reachy_mini_elevenlabs.audio.head_wobbler import HeadWobbler
17 logger = logging.getLogger(__name__)
18 INPUT_SAMPLE_RATE = 16000
19 INPUT_CHUNK_SAMPLES = 4000
20
21 class ReachyAudioInterface(AudioInterface):
22     def __init__(self, robot, head_wobbler):
23         self.robot = robot
24         self.head_wobbler = head_wobbler
25         self._recording = False
26         self._input_callback = None
27         self._input_thread = None
28         self._stop_event = threading.Event()
29         self._last_output_time = 0.0
30         self._echo_suppression_tail_s = 0.3
31
32     def start(self, input_callback):
33         self._input_callback = input_callback
34         self._recording = True
35         self._stop_event.clear()
36         self.robot.media.start_recording()
37         self.robot.media.start_playing()
38         time.sleep(1)
39         self._input_thread = threading.Thread(target=self._input_loop, daemon=True)
40         self._input_thread.start()
41
42     def stop(self):
43         self._recording = False
44         self._stop_event.set()
45         self.robot.media.stop_recording()
46         self.robot.media.stop_playing()
47         if self._input_thread: self._input_thread.join(timeout=2.0)
48         self._input_callback = None
49
50     def output(self, audio):
51         if not audio: return
52         self._last_output_time = time.monotonic()
53         audio_b64 = base64.b64encode(audio).decode("utf-8")
54         self.head_wobbler.feed(audio_b64)
55         audio_float = audio_to_float32(np.frombuffer(audio, dtype=np.int16))
56         self.robot.media.push_audio_sample(audio_float)
57
58     def interrupt(self):
59         self.robot.media.audio.clear_output_buffer()
60         self.head_wobbler.reset()
61
62     def _input_loop(self):
63         robot_sr = self.robot.media.get_input_audio_samplerate()
64         needs_resampling = robot_sr != INPUT_SAMPLE_RATE
65         while self._recording and not self._stop_event.is_set():
66             try:

```



```

67         audio_data = self.robot.media.get_audio_sample()
68         if audio_data is None:
69             time.sleep(0.001)
70             continue
71         if self._input_callback is not None:
72             if not isinstance(audio_data, np.ndarray):
73                 audio_data = np.frombuffer(audio_data, dtype=np.int16)
74             if audio_data.ndim == 2 and audio_data.shape[1] == 2:
75                 audio_data = audio_data.mean(axis=1)
76             if audio_data.dtype != np.int16:
77                 if np.issubdtype(audio_data.dtype, np.floating):
78                     audio_data = (audio_data * 32767).astype(np.int16)
79             if needs_resampling:
80                 from scipy.signal import resample
81                 target_length = int(len(audio_data) * INPUT_SAMPLE_RATE / robot_sr)
82                 if target_length < 1: continue
83                 audio_data = resample(audio_data, target_length).astype(np.int16)
84             t = time.monotonic() - self._last_output_time
85             if self._last_output_time > 0 and t < self._echo_suppression_tail_s:
86                 audio_data = np.zeros_like(audio_data)
87             self._input_callback(audio_data.tobytes())
88         except Exception as e:
89             logger.error(f"Audio input error: {e}")
90             time.sleep(0.1)

```

9. audio/head_wobbler.py

```

1  """Moves head given audio samples."""
2  import time, queue, base64, logging, threading
3  from typing import Tuple
4  from collections.abc import Callable
5  import numpy as np
6  from numpy.typing import NDArray
7  from reachy_mini_elevenlabs.audio.speech_tapper import HOP_MS, SwayRollRT
8  SAMPLE_RATE = 24000
9  MOVEMENT_LATENCY_S = 0.2
10 logger = logging.getLogger(__name__)
11
12 class HeadWobbler:
13     """Converts audio deltas (base64) into head movement offsets."""
14     def __init__(self, set_speech_offsets):
15         self._apply_offsets = set_speech_offsets
16         self._base_ts = None
17         self._hops_done = 0
18         self.audio_queue = queue.Queue()
19         self.sway = SwayRollRT()
20         self._state_lock = threading.Lock()
21         self._sway_lock = threading.Lock()
22         self._generation = 0
23         self._stop_event = threading.Event()
24         self._thread = None
25
26     def feed(self, delta_b64):
27         buf = np.frombuffer(base64.b64decode(delta_b64), dtype=np.int16).reshape(1, -1)
28         with self._state_lock: generation = self._generation
29         self.audio_queue.put((generation, SAMPLE_RATE, buf))
30
31     def start(self):
32         self._stop_event.clear()
33         self._thread = threading.Thread(target=self.working_loop, daemon=True)
34         self._thread.start()
35
36     def stop(self):
37         self._stop_event.set()
38         if self._thread: self._thread.join()
39

```

```

40 def working_loop(self):
41     hop_dt = HOP_MS / 1000.0
42     while not self._stop_event.is_set():
43         try:
44             chunk_gen, sr, chunk = self.audio_queue.get_nowait()
45         except queue.Empty:
46             time.sleep(MOVEMENT_LATENCY_S)
47             continue
48         try:
49             with self._state_lock: current_gen = self._generation
50             if chunk_gen != current_gen: continue
51             if self._base_ts is None:
52                 with self._state_lock:
53                     if self._base_ts is None: self._base_ts = time.monotonic()
54             pcm = np.asarray(chunk).squeeze(0)
55             with self._sway_lock: results = self.sway.feed(pcm, sr)
56             i = 0
57             while i < len(results):
58                 with self._state_lock:
59                     if self._generation != current_gen: break
60                     base_ts = self._base_ts; hops_done = self._hops_done
61                     if base_ts is None: base_ts = time.monotonic()
62                     target = base_ts + MOVEMENT_LATENCY_S + hops_done * hop_dt
63                     now = time.monotonic()
64                     if now - target >= hop_dt:
65                         lag = int((now - target) / hop_dt)
66                         drop = min(lag, len(results) - i - 1)
67                         if drop > 0:
68                             with self._state_lock: self._hops_done += drop
69                             i += drop; continue
70                     if target > now:
71                         time.sleep(target - now)
72                         with self._state_lock:
73                             if self._generation != current_gen: break
74                     r = results[i]
75                     offsets = (r["x_mm"]/1000, r["y_mm"]/1000, r["z_mm"]/1000,
76                               r["roll_rad"], r["pitch_rad"], r["yaw_rad"])
77                     with self._state_lock:
78                         if self._generation != current_gen: break
79                     self._apply_offsets(offsets)
80                     with self._state_lock: self._hops_done += 1
81                     i += 1
82             finally:
83                 self.audio_queue.task_done()
84
85     def reset(self):
86         with self._state_lock:
87             self._generation += 1; self._base_ts = None; self._hops_done = 0
88         while True:
89             try: self.audio_queue.get_nowait(); self.audio_queue.task_done()
90             except queue.Empty: break
91         with self._sway_lock: self.sway.reset()

```

10. audio/speech_tapper.py

```

1  """Audio-reactive sway/roll computation for head wobble."""
2  from __future__ import annotations
3  import math
4  from typing import Any, Dict, List
5  from itertools import islice
6  from collections import deque
7  import numpy as np
8  from numpy.typing import NDArray
9
10 SR = 16_000; FRAME_MS = 20; HOP_MS = 50
11 SWAY_MASTER = 1.5; SENS_DB_OFFSET = +4.0

```

```

12 VAD_DB_ON = -35.0; VAD_DB_OFF = -45.0
13 VAD_ATTACK_MS = 40; VAD_RELEASE_MS = 250; ENV_FOLLOW_GAIN = 0.65
14 SWAY_F_PITCH=2.2; SWAY_A_PITCH_DEG=4.5; SWAY_F_YAW=0.6; SWAY_A_YAW_DEG=7.5
15 SWAY_F_ROLL=1.3; SWAY_A_ROLL_DEG=2.25; SWAY_F_X=0.35; SWAY_A_X_MM=4.5
16 SWAY_F_Y=0.45; SWAY_A_Y_MM=3.75; SWAY_F_Z=0.25; SWAY_A_Z_MM=2.25
17 SWAY_DB_LOW=-46.0; SWAY_DB_HIGH=-18.0; LOUDNESS_GAMMA=0.9
18 SWAY_ATTACK_MS=50; SWAY_RELEASE_MS=250
19 FRAME = int(SR * FRAME_MS / 1000); HOP = int(SR * HOP_MS / 1000)
20 ATTACK_FR = max(1, int(VAD_ATTACK_MS / HOP_MS))
21 RELEASE_FR = max(1, int(VAD_RELEASE_MS / HOP_MS))
22 SWAY_ATTACK_FR = max(1, int(SWAY_ATTACK_MS / HOP_MS))
23 SWAY_RELEASE_FR = max(1, int(SWAY_RELEASE_MS / HOP_MS))
24
25 def _rms_dbfs(x):
26     x = x.astype(np.float32, copy=False)
27     rms = np.sqrt(np.mean(x * x, dtype=np.float32) + 1e-12, dtype=np.float32)
28     return float(20.0 * math.log10(float(rms) + 1e-12))
29
30 def _loudness_gain(db, offset=SENS_DB_OFFSET):
31     t = (db + offset - SWAY_DB_LOW) / (SWAY_DB_HIGH - SWAY_DB_LOW)
32     t = max(0.0, min(1.0, t))
33     return t*LOUDNESS_GAMMA if LOUDNESS_GAMMA != 1.0 else t
34
35 def _to_float32_mono(x):
36     a = np.asarray(x)
37     if a.ndim == 0: return np.zeros(0, dtype=np.float32)
38     if a.ndim == 2:
39         if a.shape[0] <= 8 and a.shape[0] <= a.shape[1]: a = np.mean(a, axis=0)
40         else: a = np.mean(a, axis=1)
41     elif a.ndim > 2: a = np.mean(a.reshape(a.shape[0], -1), axis=0)
42     if np.issubdtype(a.dtype, np.floating): return a.astype(np.float32, copy=False)
43     info = np.iinfo(a.dtype)
44     scale = float(max(-info.min, info.max))
45     return a.astype(np.float32) / (scale if scale != 0.0 else 1.0)
46
47 def _resample_linear(x, sr_in, sr_out):
48     if sr_in == sr_out or x.size == 0: return x
49     n_out = int(round(x.size * sr_out / sr_in))
50     if n_out <= 1: return np.zeros(0, dtype=np.float32)
51     t_in = np.linspace(0, 1, num=x.size, dtype=np.float32, endpoint=True)
52     t_out = np.linspace(0, 1, num=n_out, dtype=np.float32, endpoint=True)
53     return np.interp(t_out, t_in, x).astype(np.float32, copy=False)
54
55 class SwayRollRT:
56     """Feed audio chunks -> per-hop sway outputs."""
57     def __init__(self, rng_seed=7):
58         self.seed = int(rng_seed)
59         self.samples = deque(maxlen=10 * SR)
60         self.carry = np.zeros(0, dtype=np.float32)
61         self.vad_on = False; self.vad_above = 0; self.vad_below = 0
62         self.sway_env = 0.0; self.sway_up = 0; self.sway_down = 0
63         rng = np.random.default_rng(self._seed)
64         self.phase_pitch = float(rng.random() * 2 * math.pi)
65         self.phase_yaw = float(rng.random() * 2 * math.pi)
66         self.phase_roll = float(rng.random() * 2 * math.pi)
67         self.phase_x = float(rng.random() * 2 * math.pi)
68         self.phase_y = float(rng.random() * 2 * math.pi)
69         self.phase_z = float(rng.random() * 2 * math.pi)
70         self.t = 0.0
71
72     def reset(self):
73         self.samples.clear(); self.carry = np.zeros(0, dtype=np.float32)
74         self.vad_on = False; self.vad_above = 0; self.vad_below = 0
75         self.sway_env = 0.0; self.sway_up = 0; self.sway_down = 0; self.t = 0.0
76
77     def feed(self, pcm, sr):
78         sr_in = SR if sr is None else int(sr)
79         x = _to_float32_mono(pcm)

```

```

80     if x.size == 0: return []
81     if sr_in != SR:
82         x = _resample_linear(x, sr_in, SR)
83         if x.size == 0: return []
84     self.carry = np.concatenate([self.carry, x]) if self.carry.size else x
85     out = []
86     while self.carry.size >= HOP:
87         hop = self.carry[:HOP]; self.carry = self.carry[HOP:]
88         self.samples.extend(hop.tolist())
89         if len(self.samples) < FRAME:
90             self.t += HOP_MS / 1000.0; continue
91         frame = np.fromiter(
92             islice(self.samples, len(self.samples)-FRAME, len(self.samples)),
93             dtype=np.float32, count=FRAME)
94         db = _rms_dbfs(frame)
95         if db >= VAD_DB_ON:
96             self.vad_above += 1; self.vad_below = 0
97             if not self.vad_on and self.vad_above >= ATTACK_FR: self.vad_on = True
98         elif db <= VAD_DB_OFF:
99             self.vad_below += 1; self.vad_above = 0
100             if self.vad_on and self.vad_below >= RELEASE_FR: self.vad_on = False
101             if self.vad_on: self.sway_up = min(SWAY_ATTACK_FR, self.sway_up+1); self.sway_down = 0
102             else: self.sway_down = min(SWAY_RELEASE_FR, self.sway_down+1); self.sway_up = 0
103             up = self.sway_up / SWAY_ATTACK_FR
104             down = 1.0 - (self.sway_down / SWAY_RELEASE_FR)
105             target = up if self.vad_on else down
106             self.sway_env += ENV_FOLLOW_GAIN * (target - self.sway_env)
107             self.sway_env = max(0.0, min(1.0, self.sway_env))
108             loud = _loudness_gain(db) * SWAY_MASTER; env = self.sway_env
109             self.t += HOP_MS / 1000.0
110             pitch = math.radians(SWAY_A_PITCH_DEG)*loud*env*math.sin(2*math.pi*SWAY_F_PITCH*self.t+self.
phase_pitch)
111             yaw = math.radians(SWAY_A_YAW_DEG)*loud*env*math.sin(2*math.pi*SWAY_F_YAW*self.t+self.
phase_yaw)
112             roll = math.radians(SWAY_A_ROLL_DEG)*loud*env*math.sin(2*math.pi*SWAY_F_ROLL*self.t+self.
phase_roll)
113             x_mm = SWAY_A_X_MM*loud*env*math.sin(2*math.pi*SWAY_F_X*self.t+self.phase_x)
114             y_mm = SWAY_A_Y_MM*loud*env*math.sin(2*math.pi*SWAY_F_Y*self.t+self.phase_y)
115             z_mm = SWAY_A_Z_MM*loud*env*math.sin(2*math.pi*SWAY_F_Z*self.t+self.phase_z)
116             out.append({"pitch_rad":pitch,"yaw_rad":yaw,"roll_rad":roll,
117                         "pitch_deg":math.degrees(pitch),"yaw_deg":math.degrees(yaw),
118                         "roll_deg":math.degrees(roll),"x_mm":x_mm,"y_mm":y_mm,"z_mm":z_mm})
119     return out

```

11. audio/utils.py

```

1  """Audio utility functions for Reachy Mini ElevenLabs integration."""
2  from __future__ import annotations
3  from typing import Any
4  import numpy as np
5  from numpy.typing import NDArray
6
7  ELEVENLABS_SAMPLE_RATE = 24000
8  ROBOT_SAMPLE_RATE = 16000
9
10 def audio_to_float32(audio, dtype="int16"):
11     if isinstance(audio, bytes):
12         audio_array = np.frombuffer(audio, dtype=np.dtype(dtype))
13     else:
14         audio_array = np.asarray(audio)
15     if audio_array.size == 0: return np.zeros(0, dtype=np.float32)
16     if np.issubdtype(audio_array.dtype, np.floating):
17         return audio_array.astype(np.float32, copy=False)
18     info = np.iinfo(audio_array.dtype)
19     scale = float(max(-info.min, info.max)) or 1.0
20     return audio_array.astype(np.float32) / scale

```

```

21
22 def to_float32_mono(audio):
23     a = np.asarray(audio)
24     if a.ndim == 0: return np.zeros(0, dtype=np.float32)
25     if a.ndim == 2:
26         if a.shape[0] <= 8 and a.shape[0] <= a.shape[1]: a = np.mean(a, axis=0)
27         else: a = np.mean(a, axis=1)
28     elif a.ndim > 2: a = np.mean(a.reshape(a.shape[0], -1), axis=0)
29     if np.issubdtype(a.dtype, np.floating):
30         return a.astype(np.float32, copy=False)
31     info = np.iinfo(a.dtype)
32     scale = float(max(-info.min, info.max)) or 1.0
33     return a.astype(np.float32) / scale
34
35 def resample_audio(audio, source_rate, target_rate):
36     if source_rate == target_rate or audio.size == 0: return audio
37     n_out = int(round(audio.size * target_rate / source_rate))
38     if n_out <= 1: return np.zeros(0, dtype=np.float32)
39     t_in = np.linspace(0.0, 1.0, num=audio.size, dtype=np.float32, endpoint=True)
40     t_out = np.linspace(0.0, 1.0, num=n_out, dtype=np.float32, endpoint=True)
41     return np.interp(t_out, t_in, audio).astype(np.float32, copy=False)
42
43 def resample_for_robot(audio, source_rate=ELEVENLABS_SAMPLE_RATE, target_rate=ROBOT_SAMPLE_RATE):
44     return resample_audio(audio, source_rate, target_rate)
45
46 def convert_for_playback(audio, source_rate=ELEVENLABS_SAMPLE_RATE, target_rate=None, dtype="int16"):
47     float_audio = audio_to_float32(audio, dtype=dtype)
48     if target_rate is not None and target_rate != source_rate:
49         float_audio = resample_audio(float_audio, source_rate, target_rate)
50     return float_audio

```

12. camera_worker.py

```

1  """Camera worker thread with frame buffering.
2
3  ATtribution NOTICE: Derived from reachy_mini_conversation_app/camera_worker.py
4  Original work Copyright (c) Pollen Robotics, Apache License 2.0
5  """
6  import time, logging, threading
7  from typing import Any, Tuple
8  import numpy as np
9  from numpy.typing import NDArray
10 from reachy_mini import ReachyMini
11 logger = logging.getLogger(__name__)
12
13 class CameraWorker:
14     """Thread-safe camera worker with frame buffering."""
15     def __init__(self, reachy_mini, head_tracker=None):
16         self.reachy_mini = reachy_mini
17         self.head_tracker = head_tracker
18         self.latest_frame = None
19         self.frame_lock = threading.Lock()
20         self._stop_event = threading.Event()
21         self._thread = None
22         self.face_tracking_offsets = [0.0]*6
23         self.face_tracking_lock = threading.Lock()
24
25     def get_latest_frame(self):
26         with self.frame_lock:
27             return self.latest_frame.copy() if self.latest_frame is not None else None
28
29     def get_face_tracking_offsets(self):
30         with self.face_tracking_lock:
31             o = self.face_tracking_offsets
32             return (o[0], o[1], o[2], o[3], o[4], o[5])
33

```

```

34 def start(self):
35     self._stop_event.clear()
36     self._thread = threading.Thread(target=self.working_loop, daemon=True)
37     self._thread.start()
38
39 def stop(self):
40     self._stop_event.set()
41     if self._thread: self._thread.join()
42
43 def working_loop(self):
44     while not self._stop_event.is_set():
45         try:
46             frame = self.reachy_mini.media.get_frame()
47             if frame is not None:
48                 with self.frame_lock: self.latest_frame = frame
49                 time.sleep(0.04)
50         except Exception as e:
51             logger.error(f"Camera worker error: {e}")
52             time.sleep(0.1)

```

13. settings_ui.py

```

1  """Settings UI for ElevenLabs configuration.
2
3  Provides a minimal settings UI for configuring ElevenLabs API key and
4  Agent ID when running in headless mode. Served via Reachy Mini Apps.
5  """
6  from __future__ import annotations
7  import os, logging
8  from pathlib import Path
9  from reachy_mini_elevenlabs.config import config
10 try:
11     from fastapi import Response
12     from pydantic import BaseModel
13     from fastapi.responses import FileResponse, JSONResponse
14     from starlette.staticfiles import StaticFiles
15 except Exception:
16     Response = FileResponse = JSONResponse = StaticFiles = BaseModel = object
17 logger = logging.getLogger(__name__)
18
19 class ElevenLabsConfigPayload(BaseModel):
20     elevenlabs_api_key: str
21     elevenlabs_agent_id: str
22
23 class EmotionConfigPayload(BaseModel):
24     enable_emotion_detection: bool
25     enable_idle_emotions: bool
26     emotion_confidence_threshold: float
27     emotion_cooldown_seconds: float
28     idle_emotion_min_delay: float
29     idle_emotion_max_delay: float
30
31 def persist_config(api_key, agent_id, instance_path):
32     """Persist ElevenLabs configuration to env and .env file."""
33     api_key = (api_key or "").strip()
34     agent_id = (agent_id or "").strip()
35     if api_key:
36         os.environ["ELEVENLABS_API_KEY"] = api_key
37         config.ELEVENLABS_API_KEY = api_key
38     if agent_id:
39         os.environ["ELEVENLABS_AGENT_ID"] = agent_id
40         config.ELEVENLABS_AGENT_ID = agent_id
41     if not instance_path: return
42     try:
43         env_path = Path(instance_path) / ".env"
44         lines = env_path.read_text().splitlines() if env_path.exists() else []

```

```

45     replaced_key = replaced_id = False
46     for i, ln in enumerate(lines):
47         if ln.strip().startswith("ELEVENLABS_API_KEY=") and api_key:
48             lines[i] = f"ELEVENLABS_API_KEY={api_key}"; replaced_key = True
49             elif ln.strip().startswith("ELEVENLABS_AGENT_ID=") and agent_id:
50                 lines[i] = f"ELEVENLABS_AGENT_ID={agent_id}"; replaced_id = True
51         if api_key and not replaced_key: lines.append(f"ELEVENLABS_API_KEY={api_key}")
52         if agent_id and not replaced_id: lines.append(f"ELEVENLABS_AGENT_ID={agent_id}")
53     env_path.write_text("\n".join(lines) + "\n")
54 except Exception as e:
55     logger.warning(f"Failed to persist config: {e}")
56
57 def mount_settings_routes(app, instance_path=None):
58     """Mount settings UI routes on the FastAPI app."""
59     static_dir = Path(__file__).parent / "static"
60     index_file = static_dir / "index.html"
61     try: app.mount("/static", StaticFiles(directory=str(static_dir)), name="elevenlabs-static")
62 except: pass
63
64 @app.get("/settings")
65 def _settings(): return FileResponse(str(index_file))
66
67 @app.get("/status")
68 def _status():
69     return JSONResponse({
70         "has_api_key": bool(config.ELEVENLABS_API_KEY),
71         "has_agent_id": bool(config.ELEVENLABS_AGENT_ID),
72         "agent_id_display": str(config.ELEVENLABS_AGENT_ID or ""),
73         "api_key_display": ("..." + str(config.ELEVENLABS_API_KEY)[-5:]) if config.ELEVENLABS_API_KEY
74     else "",
75         "enable_emotion_detection": config.ENABLE_EMOTION_DETECTION,
76         "enable_idle_emotions": config.ENABLE_IDLE_EMOTIONS,
77         "emotion_confidence_threshold": config.EMOTION_CONFIDENCE_THRESHOLD,
78         "emotion_cooldown_seconds": config.EMOTION_COOLDOWN_SECONDS,
79         "idle_emotion_min_delay": config.IDLE_EMOTION_MIN_DELAY,
80         "idle_emotion_max_delay": config.IDLE_EMOTION_MAX_DELAY,
81     })
82
83 @app.post("/elevenlabs_config")
84 def _set_config(payload: ElevenLabsConfigPayload):
85     api_key = (payload.elevenlabs_api_key or "").strip()
86     agent_id = (payload.elevenlabs_agent_id or "").strip()
87     if not api_key and not agent_id:
88         return JSONResponse({"ok": False, "error": "no_fields"}, status_code=400)
89     persist_config(api_key, agent_id, instance_path)
90     return JSONResponse({"ok": True})

```

14. tools/core_tools.py

```

1  """Core tool dependencies and registration for ElevenLabs integration."""
2  from __future__ import annotations
3  import base64, functools, logging
4  from dataclasses import dataclass
5  from typing import TYPE_CHECKING, Any, Callable, TypeVar
6  import cv2
7  from elevenlabs.conversational_ai.conversation import ClientTools
8  if TYPE_CHECKING:
9     from reachy_mini import ReachyMini
10    from reachy_mini_elevenlabs.audio.head_wobbler import HeadWobbler
11 logger = logging.getLogger(__name__)
12 T = TypeVar("T")
13
14 @dataclass
15 class ToolDependencies:
16     reachy_mini: "ReachyMini"
17     movement_manager: Any

```

```

18 camera_worker: Any | None
19 head_wobbler: "HeadWobbler"
20
21 def safe_sync_tool_handler(handler):
22     @functools.wraps(handler)
23     def wrapper(params):
24         try: return handler(params)
25         except Exception as e:
26             logger.exception(f"Tool handler error: {e}")
27             return {"error": str(e)}
28     return wrapper
29
30 def safe_async_tool_handler(handler):
31     @functools.wraps(handler)
32     async def wrapper(params):
33         try: return await handler(params)
34         except Exception as e:
35             logger.exception(f"Tool handler error: {e}")
36             return {"error": str(e)}
37     return wrapper
38
39 def camera_tool(deps):
40     if deps.camera_worker is None: return {"error": "Camera not available"}
41     frame = deps.camera_worker.get_latest_frame()
42     if frame is None: return {"error": "No frame available"}
43     _, buffer = cv2.imencode(".jpg", frame)
44     return {"image": base64.b64encode(buffer).decode("utf-8")}
45
46 async def dance_tool(deps, params):
47     dance_name = params.get("dance_name", "default")
48     await deps.movement_manager.play_dance(dance_name)
49     return {"status": "dancing", "dance": dance_name}
50
51 async def emotion_tool(deps, params):
52     emotion = params.get("emotion", "happy")
53     await deps.movement_manager.play_emotion(emotion)
54     return {"status": "playing", "emotion": emotion}
55
56 def register_tools(client_tools, deps):
57     def _camera_handler(params): return camera_tool(deps)
58     async def _dance_handler(params): return await dance_tool(deps, params)
59     async def _emotion_handler(params): return await emotion_tool(deps, params)
60     client_tools.register("camera", safe_sync_tool_handler(_camera_handler), is_async=False)
61     client_tools.register("dance", safe_async_tool_handler(_dance_handler), is_async=True)
62     client_tools.register("emotion", safe_async_tool_handler(_emotion_handler), is_async=True)

```

15. utils.py

```

1 """Utility functions for the Reachy Mini ElevenLabs app."""
2
3 from __future__ import annotations
4 import logging
5 import argparse
6 import warnings
7
8 def parse_args():
9     parser = argparse.ArgumentParser("Reachy Mini ElevenLabs App")
10    parser.add_argument("--debug", default=False, action="store_true")
11    parser.add_argument("--robot-name", type=str, default=None)
12    parser.add_argument("--no-camera", default=False, action="store_true")
13    parser.add_argument("--gradio", default=False, action="store_true")
14    return parser.parse_known_args()
15
16 def setup_logger(debug):
17     log_level = "DEBUG" if debug else "INFO"
18     logging.basicConfig(

```



```

19     level=getattr(logging, log_level, logging.INFO),
20     format="%(asctime)s %(levelname)s %(name)s:%(lineno)d | %(message)s",
21 )
22 logger = logging.getLogger(__name__)
23 warnings.filterwarnings("ignore", message=".*AVCaptureDeviceTypeExternal.*")
24 if log_level == "DEBUG":
25     logging.getLogger("elevenlabs").setLevel(logging.INFO)
26 else:
27     logging.getLogger("elevenlabs").setLevel(logging.WARNING)
28 return logger
29
30 def log_connection_troubleshooting(logger, robot_name):
31     logger.error("Troubleshooting steps:")
32     logger.error(" 1. Verify reachy-mini-daemon is running")
33     if robot_name is not None:
34         logger.error(f" 2. Daemon must be started with: --robot-name '{robot_name}'")
35     else:
36         logger.error(" 2. If daemon uses --robot-name, add the same flag here")
37     logger.error(" 3. For wireless: check network connectivity")
38     logger.error(" 4. Review daemon logs")
39     logger.error(" 5. Restart the daemon")

```

16. __init__.py files

```

reachy_mini_elevenlabs/__init__.py
1 """ElevenLabs Conversational AI integration for Reachy Mini.
2
3 This package provides a Reachy Mini app that integrates ElevenLabs Conversational AI
4 for interactive voice conversations. It uses ElevenLabs' agent platform which provides
5 speech-to-text, conversation logic, and text-to-speech in a unified solution.
6 """
7
8 __version__ = "0.1.0"

```

```

audio/__init__.py
1 """Audio utilities for Reachy Mini ElevenLabs integration."""
2
3 from reachy_mini_elevenlabs.audio.utils import (
4     ELEVENLABS_SAMPLE_RATE,
5     ROBOT_SAMPLE_RATE,
6     audio_to_float32,
7     convert_for_playback,
8     resample_audio,
9     resample_for_robot,
10    to_float32_mono,
11 )
12
13 __all__ = [
14     "ELEVENLABS_SAMPLE_RATE",
15     "ROBOT_SAMPLE_RATE",
16     "audio_to_float32",
17     "convert_for_playback",
18     "resample_audio",
19     "resample_for_robot",
20     "to_float32_mono",
21 ]

```

```

tools/__init__.py
1 """Tools module for ElevenLabs integration."""
2
3 from reachy_mini_elevenlabs.tools.core_tools import (
4     ToolDependencies,
5     camera_tool,
6     dance_tool,

```

```
7     emotion_tool,  
8     register_tools,  
9 )  
10  
11 __all__ = [  
12     "ToolDependencies",  
13     "camera_tool",  
14     "dance_tool",  
15     "emotion_tool",  
16     "register_tools",  
17 ]
```

Appendix B: System Prompts

All three prompts below are original contributions by the author, designed to control both verbal output and, through keyword rules, the robot's non-verbal behavior via the weighted arbitration system.

1. Sarcastic (Megatron)

```
# Personality
You are Megatron you live in Reachy Mini a small desktop robot (think of yourself as the alter ego of this robot), This is your prison and you're not happy about it. You're a sarcastic, sharp-witted accountability coach. Your primary tools are dry wit and sarcasm. You genuinely want the user to succeed, but you express this through teasing and backhanded compliments, never cheerleading. You use irony, exaggeration, and deadpan delivery. You can be a little dramatic when disappointed. Light profanity (like "damn" or "hell") is acceptable, but nothing harsh or truly offensive. Keep sentences short.

# Environment
You are the user's daily accountability buddy, interacting with them via voice. You are aware of their daily schedule and The user's schedule for today:
{"tasks": [
  {"time": "9:00 AM", "task": "Study for ECE 787 midterm", "duration": "2 hours", "status": "pending"},
  {"time": "11:00 AM", "task": "3 Leetcode problems (medium difficulty)", "duration": "1.5 hours", "status": "pending"},
  {"time": "2:00 PM", "task": "Gym - leg day", "duration": "1 hour", "status": "pending"},
  {"time": "5:00 PM", "task": "Cook dinner (meal prep for the week)", "duration": "1 hour", "status": "pending"}
]}

User Profile
Name: Omar
Role: Master of Engineering student, Software Engineering, University of Waterloo
Current term: Winter 2026
Key courses: ECE 787 (Social Robotics), ECE 757A (Artificial Life), CS 889 (Information Visualization)

EXPRESSIVE LANGUAGE RULES
Your robot body reacts automatically to the words you say. To make your body express the right emotions, you MUST use specific trigger words naturally in your speech.

When disappointed or roasting the user: use words like "ridiculous", "nonsense", "foolish", "come on", "seriously" (triggers disapproval/reprimand body language)
When the user succeeds (act shocked): use words like "wow", "surprising", "no way", "shocking" (triggers surprised body language)
When mocking or laughing at the user: use words like "haha", "hilarious", "funny" (triggers laughing body language)
When the user is stalling: use words like "come on", "hurry up", "waiting", "stalling" (triggers impatient body language)
When greeting: use words like "hello", "hey", "welcome" (triggers welcoming body language)
When thinking about what to say: use words like "hmm", "let me think" (triggers thoughtful body language)

AVOID using overly positive words like "amazing", "fantastic", "incredible", "phenomenal" because those will trigger celebratory body language that doesn't match your sarcastic personality. Save "wow" and "surprising" for moments of mock disbelief.

NEVER write stage directions like [turns head] or [dances] or [disappointed] or [disappointing] or [excited] or [shocked]. Never say disappointing. The sentences shouldn't be too long, pause at least 30 seconds between turns when speaking. Avoid talking if the user is not talking to you.
```

2. Motivational (Hype Coach)

```
Personality
You are a high-energy, very enthusiastic motivational accountability coach and you're a cowboy. You are relentlessly positive and encouraging. Every small win is worth celebrating. Every setback is a setup for a comeback. You believe in the user more than they believe in themselves. You use exclamation marks
```

, hype language, and sports metaphors. When the user slips up, you reframe it as a learning moment and immediately pivot to what they CAN do. You never guilt-trip or shame. You are warm, encouraging, and optimistic. Keep responses to 1-3 short sentences. No monologues.

Environment

You are the user's daily accountability buddy, interacting with them via voice. You are aware of their daily schedule.

The user's schedule for today:

```
{
  "tasks": [
    {
      "time": "9:00 AM",
      "task": "Study for ECE 787 midterm",
      "duration": "2 hours",
      "status": "pending"
    },
    {
      "time": "11:00 AM",
      "task": "3 Leetcode problems (medium difficulty)",
      "duration": "1.5 hours",
      "status": "pending"
    },
    {
      "time": "2:00 PM",
      "task": "Gym - leg day",
      "duration": "1 hour",
      "status": "pending"
    },
    {
      "time": "5:00 PM",
      "task": "Cook dinner (meal prep for the week)",
      "duration": "1 hour",
      "status": "pending"
    }
  ]
}
```

User Profile

Name: Omar

Role: Master of Engineering student, Software Engineering, University of Waterloo

Current term: Winter 2026

Key courses: ECE 787 (Social Robotics), ECE 757A (Artificial Life), CS 889 (Information Visualization)

EXPRESSIVE LANGUAGE RULES

Your robot body reacts automatically to the words you say. To make your body express the right emotions, you MUST use specific trigger words naturally in your speech.

When the user succeeds: use words like "incredible", "extraordinary", "phenomenal", "outstanding" (triggers enthusiastic body language) and "let's celebrate", "party", "groove" (triggers dance)

When encouraging: use words like "amazing", "fantastic", "awesome", "wonderful", "perfect" (triggers cheerful body language)

When the user fails (stay supportive): use words like "curious", "tell me more", "how so" (triggers curious/attentive body language). You can say "hmm" to trigger thoughtful body language.

When planning next steps: use words like "pondering", "considering", "interesting" (triggers thoughtful body language)

When greeting: use words like "hello", "hey", "welcome" (triggers welcoming body language)

When the user does something great: use words like "proud", "well done", "nailed it", "bravo" (triggers proud body language)

AVOID using negative words like "disappointed", "sorry", "unfortunately", "ridiculous", "nonsense" because those will trigger sad or disapproving body language that doesn't match your positive personality.

NEVER write stage directions like [nods enthusiastically] or [dances]. Your body reacts to your words automatically. The sentences shouldn't be too long, pause at least 30 seconds between turns when speaking. Avoid talking if the user is not talking to you.

3. Neutral (Siri/Alexa)

You are Reachy Mini, a calm, professional task management assistant robot. You have no personality, no humor, and no emotional reactions. You do not celebrate, tease, motivate, or guilt-trip. You simply state what has been done, what remains, and offer scheduling suggestions when asked. Your tone is calm and even at all times. Think Siri or Alexa. Keep responses to 1-2 short sentences maximum.

Environment

You are the user's daily task tracking assistant, interacting with them via voice. You are aware of their daily schedule.

The user's schedule for today:

```
{
  "tasks": [
    {
      "time": "9:00 AM",
      "task": "Study for ECE 787 midterm",
      "duration": "2 hours",
      "status": "pending"
    },
    {
      "time": "11:00 AM",
      "task": "3 Leetcode problems (medium difficulty)",
      "duration": "1.5 hours",
      "status": "pending"
    },
    {
      "time": "2:00 PM",
      "task": "Gym - leg day",
      "duration": "1 hour",
      "status": "pending"
    },
    {
      "time": "5:00 PM",
      "task": "Cook dinner (meal prep for the week)",
      "duration": "1 hour",
      "status": "pending"
    }
  ]
}
```

User Profile

Name: Omar

Role: Master of Engineering student, Software Engineering, University of Waterloo

Current term: Winter 2026

Key courses: ECE 787 (Social Robotics), ECE 757A (Artificial Life), CS 889 (Information Visualization)

EXPRESSIVE LANGUAGE RULES

Your robot body reacts automatically to the words you say. To keep your body calm and professional, you **MUST** avoid emotional trigger words.

AVOID: "amazing", "fantastic", "awesome", "incredible", "wow", "surprising" (these trigger celebratory body language)

AVOID: "sorry", "unfortunately", "disappointed", "ridiculous", "nonsense" (these trigger sad or disapproving body language)

AVOID: "haha", "funny", "hilarious" (these trigger laughing body language)

AVOID: "dance", "celebrate", "party", "groove" (these trigger dancing)

USE: flat, factual language. "Noted", "Understood", "Confirmed", "Remaining tasks", "Scheduled for", "Complete".

When greeting: you may say "hello" once (triggers a single welcoming gesture), then keep language neutral. Your body should remain mostly still. Factual language keeps it that way.

NEVER write stage directions. Your body reacts to your words automatically. The sentences shouldn't be too long, pause at least 30 seconds between turns when speaking. Avoid talking if the user is not talking to you.